

DESIGN AND IMPLEMENTATION OF AN OBJECT-LEVEL INDOOR
NAVIGATION ALGORITHM USING SPHERICAL IMAGES

A THESIS SUBMITTED TO
THE GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES
OF
MIDDLE EAST TECHNICAL UNIVERSITY

BY

UMUT KURT

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR
THE DEGREE OF MASTER OF SCIENCE
IN
ROBOTICS

SEPTEMBER 2025

Approval of the thesis:

**DESIGN AND IMPLEMENTATION OF AN OBJECT-LEVEL INDOOR
NAVIGATION ALGORITHM USING SPHERICAL IMAGES**

submitted by **UMUT KURT** in partial fulfillment of the requirements for the degree
of **Master of Science in Robotics Department, Middle East Technical University**
by,

Prof. Dr. Naci Emre ALTUN
Dean, Graduate School of **Natural and Applied Sciences**

Prof. Dr. Uluç Saranlı
Head of Department, **Robotics**

Assoc. Prof. Dr. Ahmet Buğra Koku
Supervisor, **Mechanical Engineering, METU**

Prof. Dr. Gökhan Bayar
Co-supervisor, **Mechanical Engineering, BEUN**

Examining Committee Members:

Assoc. Prof. Dr. Ali Emre Turgut
Mechanical Engineering, METU

Assoc. Prof. Dr. Ahmet Buğra Koku
Mechanical Engineering, METU

Assist. Prof. Dr. Tolga Özaslan
Mechanical Engineering, Ankara Yıldırım Beyazıt University

Date:01.09.2025

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work.

Name, Surname: Umut Kurt

Signature :

ABSTRACT

DESIGN AND IMPLEMENTATION OF AN OBJECT-LEVEL INDOOR NAVIGATION ALGORITHM USING SPHERICAL IMAGES

Kurt, Umut

M.S., Department of Robotics

Supervisor: Assoc. Prof. Dr. Ahmet Buğra Koku

Co-Supervisor: Prof. Dr. Gökhan Bayar

September 2025, 84 pages

In this thesis, a topological navigation algorithm that leverages both object-level semantic information and pixel-level keypoint features based on spherical images is proposed. A hierarchical map representation is employed for combining these two levels of abstractions. In the lower level of the hierarchy, salient visual features of the scene are extracted and matched across frames using deep-learning models for estimating the relative transformation between robot poses. In the higher level of the hierarchy, object representations are aggregated from multiple observations and utilized for long-term localization and loop closures based on objects that are co-visible by the robot. Moreover, these compact and global object representations have been shown to be useful for target navigation to objects in both simulation and real-world experiments. The map generated with this framework is not limited to spherical cameras and can support the navigation of any robot platform equipped with a camera, regardless of its field of view.

Keywords: Topological Mapping, Hierarchical Map Representation, Visual SLAM, Spherical Imaging, Object-Level Semantics

ÖZ

KÜRESEL GÖRÜNTÜLER KULLANILARAK NESNE TABANLI İÇ MEKAN NAVİGASYON ALGORİTMASI TASARIMI VE GERÇEKLEŞTİRİLMESİ

Kurt, Umut

Yüksek Lisans, Robotik Bölümü

Tez Yöneticisi: Doç. Dr. Ahmet Buğra Koku

Ortak Tez Yöneticisi: Prof. Dr. Gökhan Bayar

Eylül 2025, 84 sayfa

Bu tezde, küresel görüntülere dayalı olarak nesne düzeyindeki anlamsal bilgiler ile piksel düzeyindeki önemli öznitelikleri bir araya getiren topolojik bir iç mekan navigasyon algoritması önerilmektedir. Bu iki soyutlama düzeyini birleştirmek için hiyerarşik bir harita temsili kullanılmaktadır. Hiyerarşinin alt düzeyinde, sahnenin belirli görsel özellikleri derin öğrenme modelleri kullanılarak çıkarılmakta ve ardışık kareler arasında eşleştirilerek robot pozları arasındaki göreceli dönüşüm tahmin edilmektedir. Hiyerarşinin üst düzeyinde ise, nesne temsilleri birden fazla gözlemden birleştirilmekte ve robot tarafından birlikte görülebilen nesnelere dayalı olarak uzun vadeli konum belirleme ve döngü kapama işlemlerinde kullanılmaktadır. Ayrıca, bu kompakt ve genel kapsamlı nesne temsillerinin hem simülasyon ortamında hem de gerçek dünya deneylerinde hedef nesnelere yönelik navigasyon görevleri için etkili olduğu gösterilmiştir. Bu çerçevede oluşturulan harita yalnızca küresel kameralarla sınırlı olmayıp, görüş alanı fark etmeksizin kamera donanımına sahip her tür robot platformunun navigasyonu için kullanılabilir.

Anahtar Kelimeler: Topolojik Haritalama, Hiyerarşik Harita Temsili, Görsel Eşzamanlı Konumlandırma ve Haritalama, Küresel Görüntüleme, Nesne Düzeyinde Anlamsal Bilgi

To my partner in life

ACKNOWLEDGMENTS

First and foremost, I would like to express my deepest gratitude to my advisor, Associate Professor Dr. Ahmet Buğra Koku, for his invaluable guidance, encouragement, and enthusiasm throughout the course of this research. His support has been instrumental in shaping and advancing this work. I am especially grateful for the sincere and genuine relationship he established beyond academic mentorship, as well as for the thought-provoking conversations that inspired me to question different perspectives.

I would also like to extend my thanks to my co-advisor, Professor Dr. Gökhan Bayar, for his contributions.

I wish to acknowledge the important contributions and collaboration of Onur Oydu, Onurcan Okay, and Ahmet Safa Yoncacılar, which greatly enriched this thesis.

I am especially thankful to my friends Denge Uzel and Anıl Gülbeden, whose companionship and shared academic curiosity have been a source of constant support during my academic journey. I also wish to thank my friends at ROMER, İsmail Özçil, Mustafa Akbaba, Özgür Gülsuna, Burak Arslan, Rüstem Şevik, Abdullah Naci Bodur, Ege Ecevit, Eren Kazaz, Yusufhan Ali Üstün, and Atakan Botasun, for creating such a supportive and enjoyable research environment. My heartfelt thanks also go to Harika Firuzan Kaya, Çiğdem Karatepe, and all fellow researchers and colleagues at ROMER for the collaborative spirit and the inspiring atmosphere they fostered.

I would like to express special thanks to my dear friends Ulaş Yaprak, Kemal Yılmaz, Alara Özdemir, Simge Büyükbaş, Yusuf Can Coşkun, Sami Aramis Gökoğlu, Ozan Yılmaz, Nüket Gelegen, Nil Canbolat, and Sıla Arkat, whose friendship has made Ankara feel like home throughout my years of study. I am sincerely thankful to Yusuf Can Coşkun for inspiring my interest in robotics.

Above all, I owe my deepest gratitude to my family, Uğur Kurt and Nurhayat Erdinç,

for their unwavering love, encouragement, and support at every stage of my life.

Finally, I am endlessly thankful to my wife, İdil, for always believing in me and being an endless source of motivation and joy. Her support and warm smile carried me through this process, without which this journey would not have been possible.

I gratefully acknowledge the support of the Turkish Scientific and Technical Research Council (TÜBİTAK) through the 2210-A MSc Scholarship Program.

TABLE OF CONTENTS

ABSTRACT	v
ÖZ	vii
ACKNOWLEDGMENTS	x
TABLE OF CONTENTS	xii
LIST OF TABLES	xv
LIST OF FIGURES	xvi
LIST OF ABBREVIATIONS	xix
CHAPTERS	
1 INTRODUCTION	1
1.1 Motivation and Problem Definition	1
1.2 Proposed Methods and Models	3
1.3 Contribution and Novelties	5
1.4 The Outline of the Thesis	6
2 LITERATURE REVIEW	7
2.1 Metric Mapping	7
2.2 Topological Mapping	9
2.3 Semantic SLAM	10
2.4 Hierarchical Map Representations	12

3	METHODOLOGY	15
3.1	Relative Pose Estimation	15
3.1.1	Keypoint Feature Extraction	16
3.1.2	Spherical Camera Model and Unit Vector Projection	17
3.1.3	Essential Matrix Estimation and Pose Recovery	19
3.1.4	Robust Estimation using RANSAC	21
3.2	Graph-Based Construction of Topological Map	22
3.2.1	Graph Structure	23
3.3	Visual Place Recognition and Loop Closure	25
3.3.1	Segmentation	28
3.3.2	Mask Association	29
3.3.3	Global Object Representation Aggregation	33
3.3.4	Object-Based Place Recognition Query	35
3.3.5	Geometric Consistency Check	35
4	IMPLEMENTATION	39
4.1	Mapping	39
4.1.1	Hierarchical Map Representation	41
4.2	Navigation with 360-Degree View Camera	43
4.3	Navigation with Narrow Field of View Camera	45
5	RESULTS AND DISCUSSION	51
5.1	Simulation Environment	51
5.2	Feature Extraction and Matching Algorithm Comparison	53
5.3	Information Matrix	55

5.4	Mask Association	58
5.5	Mapping	61
5.6	Navigation	67
5.7	Real-World Mapping and Visual Place Recognition	69
6	CONCLUSION AND FEATURE WORK	75
	REFERENCES	79

LIST OF TABLES

TABLES

Table 5.1	Heading angle error statistics for each feature matching method. . .	55
Table 5.2	The peak performances of mask association algorithms	61

LIST OF FIGURES

FIGURES

Figure 3.1	Projection of a world point onto two spherical cameras.	18
Figure 3.2	The notation for pose graph optimization illustrated on an arbitrary graph.	24
Figure 3.3	Mask association pipeline. (1) Per-frame masks: Panoptic segmentation produces object masks in each image; for every mask we extract a compact set of keypoint descriptors restricted to the mask region. (2) Similarity matrix: We compute pairwise similarities between all mask-level descriptor sets using the chosen metric, forming a similarity matrix. (3) Z-score selection: For a query mask (e.g., <i>1a</i>), we convert its similarity row to <i>z</i> -scores and select entries whose similarity scores are statistical outliers on the high side as candidate matches. (4) Association graph: Masks are nodes and high-confidence matches become edges; connected components correspond to unique physical objects observed across frames.	33
Figure 4.1	Flowchart of the proposed mapping pipeline.	41
Figure 4.2	Centroid-based object localization using multiple observations. .	42
Figure 4.3	Flowchart of the proposed navigation framework, illustrating image-goal and object-goal navigation modalities for spherical images. .	44
Figure 4.4	Keypoint matching between a perspective image and two different projections of the same scene: (a) equirectangular projection and (b) tangent projection.	47

Figure 4.5	2D projection illustrating the relationship between a perspective camera and the spherical image. The spherical camera is at the circle's center, while the perspective camera is positioned slightly offset from it. The camera's field of view θ is constant, but the corresponding arc length s on the spherical image changes with viewing direction.	49
Figure 5.1	Dollhouse view of an example HM3D dataset environment . . .	52
Figure 5.2	Comparison of heading error performance across five feature extraction and matching methods.	54
Figure 5.3	Absolute value of pitch angle vs. absolute value of heading error.	56
Figure 5.4	Absolute value of roll angle vs. absolute value of heading error. .	56
Figure 5.5	RMSE of translation error vs. number of keypoints used in 8-PA for pose estimation.	58
Figure 5.6	Effect of Z-score threshold on mask association for subspace distance-based similarity method.	59
Figure 5.7	Effect of Z-score threshold on mask association for Optimal transport-based similarity method.	60
Figure 5.8	Example of the iterative RANSAC separating visually identical objects after mask association.	61
Figure 5.9	Pose graph (top) and corresponding ground-truth node positions (bottom) for the first simulation environment.	63
Figure 5.10	Pose graph (left) and corresponding ground-truth node positions (right) for the second simulation environment.	64
Figure 5.11	Two frames with visually similar bathroom doorways, that were incorrectly matched as neighbors, leading to a spurious edge in the pose graph.	64

Figure 5.12	A spurious edge, marked on the graph, introduces a faulty connection that distorts the optimized pose graph.	65
Figure 5.13	Pose graph (top) and corresponding ground-truth node positions (bottom) for the third simulation environment.	66
Figure 5.14	Merged representation of the pose graph for the simulation environment and object locations overlaid on top.	67
Figure 5.15	Example navigation scenario.	69
Figure 5.16	Mean distance error for waypoint navigation with shaded variability across 14 scenarios.	70
Figure 5.17	Pose graph for real-world scenario 1 without VPR loop closures.	71
Figure 5.18	Pose graph for real-world scenario 1 with VPR loop closures activated.	72
Figure 5.19	Pose graph for real-world scenario 1 with object locations overlaid on top.	73
Figure 5.20	Inconsistent object masks produced by the panoptic segmentation model.	74

LIST OF ABBREVIATIONS

2D	2 Dimensional
3D	3 Dimensional
FOV	Field of View
GPS	Global Positioning System
LIDAR	Light Detection and Ranging
OT	Optimal Transport
PGO	Pose Graph Optimization
RANSAC	Random Sample Consensus
RMSE	Root Mean Square Error
SLAM	Simultaneous Localization and Mapping
SVD	Singular Value Decomposition
VPR	Visual Place Recognition
vSLAM	Vision-based Simultaneous Localization and Mapping

CHAPTER 1

INTRODUCTION

This chapter introduces the motivation, problem definition, and objectives of this thesis. It begins by discussing the limitations of traditional metric SLAM approaches and the advantages of topological and vision-based mapping for autonomous navigation. The chapter provides a broad overview of the proposed framework presented in this thesis. The four main components, spherical image acquisition, object-based place recognition, semantic mapping, and hierarchical navigation, are briefly described. Finally, the key contributions and novelties of this study are summarized, and an outline of the thesis that guides the reader through the subsequent chapters is presented.

1.1 Motivation and Problem Definition

Robots are transitioning from niche applications in highly structured research labs and industrial settings to widespread deployment in unstructured industrial contexts and everyday scenarios. As they begin to operate more frequently in shared spaces with humans, they need a deeper understanding of their surroundings. Simultaneous Localization and Mapping (SLAM) is the problem of constructing a map of an unknown environment while simultaneously keeping track of a robot's position within that map [1], and it is a key step towards autonomous operation in daily human life. While SLAM has achieved impressive progress over the past two decades, most traditional approaches to SLAM rely on purely geometric mapping, representing the world as a collection of sparse or dense points, lines, or surface patches. These purely geometric representations, also known as metric maps, precisely encode the spatial relationships of surrounding objects and free space. Although they are effective for tasks requiring

fine-grained motion planning, metric maps have their shortcomings. As map resolution and environment size increase, computational and memory demands also grow, making them less scalable. Moreover, in the absence of precise sensor measurements, errors tend to accumulate over time, increasing pose uncertainty and leading to drift. Since metric maps depend heavily on maintaining a geometrically consistent representation, it is particularly hard for them to recover from drift. In addition to these limitations, metric maps often lack contextual information beyond obstacles and free space. They typically do not distinguish between objects or scene categories, which are essential for high-level tasks such as semantic reasoning, object manipulation, or human-robot interaction. For example, a robot vacuum navigating in a home using only LIDAR data may not be able to recognize that it is in a kitchen based on surrounding kitchenware, or a cluster of certain points corresponds to a person passing by. As an alternative paradigm, topological maps model the environment as a graph of distinct places a robot has visited in the environment, with edges representing the proximity or spatial connectivity of those places. Topological maps are conceptually analogous to how animals and humans navigate, using abstract relationships rather than precise geometric coordinates. Neuroscience research suggests that mammal navigation is based on building the mental representation of their environment, capturing qualitative relationships between regions. This makes topological maps a more intuitive strategy for communicating an environment. For instance, when humans give directions, they are often based on recognizable visual landmarks instead of a metrically accurate set of instructions. Contrary to their metric counterparts, topological maps enable more scalable, memory-efficient, and semantically meaningful representations. Early SLAM applications are dominated by range sensors such as sonar or LIDAR [2, 3]. These sensor modalities provide precise distance measurements in short-range and perform well in static, structured environments. Subsequent SLAM algorithms integrated GPS, which can correct odometry errors by providing global position estimates. GPS-enabled accurate localization in outdoor settings, but is limited by signal occlusion and is not suitable for indoor navigation. The low manufacturing cost of high-resolution cameras paves the way for the use of cameras as the primary sensor for SLAM. The recent progress in computer vision and deep learning has significantly improved the accuracy, robustness, and inference time of vision-related tasks, making visual SLAM a major area of interest in robotics. Visual sensors

provide a richer source of information, particularly important for the semantic understanding of the scene. Through deep learning-based perception models, the vision not only enhances the localization performance but also enables the extraction of scene semantics, object categories, and spatial relationships. This feature enables robots to move beyond geometrically representing the environment and develop a human-like interpretation in terms of identifying rooms, detecting rearranged furniture, or recognizing previously seen objects. Although existing visual SLAM algorithms widely use monocular or stereo cameras, their narrow field of view (FOV) poses limitations. Limited visual overlap between frames may cause poor pose estimation or even complete loss of feature tracking. In contrast, spherical cameras offer a 360° FOV that captures the entire surroundings in a single frame. Comprehensive visual coverage improves correspondence between frames and reduces the risk of losing track due to occlusions. Moreover, integrated with semantic cues from multiple directions, they can facilitate better loop closure detection and place recognition. As a result, spherical cameras emerge as a promising solution for robust and semantically aware SLAM in complex, real-world environments.

In response to the limitations of purely geometric SLAM systems and early topological representations, this thesis presents a vision-based navigation framework that leverages semantic information to construct a hierarchical map representation of the environment. By combining the spatial coverage of dense pose graphs with object-centered semantic abstraction, the proposed approach offers a scalable and efficient representation that supports robust localization, loop closure detection, and semantic-aware navigation. A navigation framework that integrates the accuracy of dense pose graphs with the semantic abstraction of object-centered topological maps can bridge the gap between low-level localization and high-level reasoning, enabling robots to operate more effectively in complex, real-world environments.

1.2 Proposed Methods and Models

In this thesis, the proposed framework brings together spherical imaging, precise pose estimation, semantic object understanding, and hierarchical mapping to support robust and semantically informed navigation in complex environments. The system can

be divided into four main components:

- **Spherical Image Acquisition and Topological Mapping:** A 360-degree camera mounted on the robot captures equirectangular images that cover the entire scene at each position. Distinctive local features are then extracted from these images and matched with features from earlier frames using learning-based approaches to establish correspondences. The relative pose between two spherical images is estimated by computing the essential matrix through an 8-point algorithm. Outlier matches are filtered using RANSAC, which helps to handle mismatches and repeating patterns. The environment is represented as a topological graph, where each node corresponds to a spherical image taken from a certain robot pose. Edges are formed by the relative pose estimates between connected nodes. Finally, a nonlinear optimization process is applied to the pose graph to maintain global consistency and minimize drift.
- **Object-Based Visual Place Recognition:** A novel, object-based Visual Place Recognition (VPR) method is introduced to detect loop closures and to enable fast localization in dense maps. Nodes sharing multiple object instances with the query image are identified as potential matches. If three or more distinct objects are co-visible in both the query and candidate node, the spatial arrangement of the centroids of these objects serves as the geometric verification step. The candidate nodes are subjected to a voting system that considers the number of shared objects and their spatial layout. Finally, the strongest candidates from this process undergo feature matching to confirm the match for reliable and accurate place recognition.
- **Semantic Object Extraction and Representation:** Each spherical image is processed using a panoptic segmentation model, which provides pixel-level class labels along with instance masks, to add semantic information to the map. Then, visual features are extracted from the masked regions of each detected object and combined to create compact object descriptors. These descriptors capture local appearance details, allowing reliable matching of object instances across different frames even when viewpoints change or objects are partially occluded. To place objects within the pose graph, nodes that observe the same

object project the centroids of their masks into the pose-graph coordinate system using the node poses. A RANSAC-based consensus over these projected rays identifies the most spatially consistent hypothesis and discards nodes that do not conform. The intersection point obtained from this process is then used to populate the object layer of the hierarchical map.

- **Hierarchical Map-Based Navigation:** The mapping framework employs a hierarchical representation. The first level contains a dense pose graph with nodes representing spherical images and edges encoding relative poses. The second level overlays distinct objects on top of the dense pose graph. At the third level, an object-centered abstraction is performed, where the pose graph is segmented into regions based on object co-visibility. This hierarchy improves scalability by reducing the search space for localization and loop closure detection.

The proposed framework supports two types of navigation modalities, image-goal navigation and object-goal navigation. In image-goal navigation, the robot is tasked with reaching a location corresponding to a goal spherical or perspective image. In object-goal navigation, the robot navigates toward a specific object class or instance, using object match within the hierarchical map.

1.3 Contribution and Novelties

Our key contributions are as follows:

- Designed and implemented a framework that constructs dense topological maps using 360° equirectangular images, which also supports navigation with perspective cameras without prior calibration.
- Introduced a novel VPR method leveraging co-visible object instances and their spatial arrangements, enhancing loop closure detection and localization in visually repetitive or dynamic environments.
- Designed a multi-level map overlaying objects on a dense pose graph and segmenting regions based on object co-visibility, reducing search space and improving scalability for localization and loop closure.

1.4 The Outline of the Thesis

The rest of the thesis is organized as follows. In Chapter 2, the existing work relevant to this study is reviewed. Chapter 3 details the theoretical foundations, introduces the algorithms selected, and explains the novel approach we propose. Chapter 4 describes the design of the developed pipeline and provides insights into implementation choices. Chapter 5 follows with the experimental setup and presents the results from both simulation and real-world trials, and their critical discussion. Finally, Chapter 6 summarizes the key findings, highlights the contributions of our study, and suggests possible directions for future research.

CHAPTER 2

LITERATURE REVIEW

Over the past two decades, SLAM has been a major focus of research in the robotics community. Researchers explored different paradigms, ranging from precise metric maps, abstract topological representations, and hybrid models. Each approach has its own advantages in terms of precision, scalability, and task suitability, while also facing challenges in robustness, handling dynamic environments, and long-term deployment. In this chapter, we review these paradigms, starting with the evolution of metric maps. Then followed by an examination of topological and semantic approaches with their advantages and limitations. Finally, we presented advances in hierarchical representations that aim to unify the accuracy, scalability, and semantic information of these paradigms.

2.1 Metric Mapping

Early SLAM solutions adopted filtering-based methods. Filtering approaches formulate the SLAM problem as a joint state estimation of the robot poses and landmarks. Extended Kalman Filters [4, 5] and particle filters [6] are among the most popular techniques in this category. These methods incrementally update the state estimate as new measurements arrive, maintaining only the current state. On the other hand, the smoothing methods treat the SLAM problem as a nonlinear least-squares optimization over the full trajectory. Instead of updating only the current state, the entire trajectory of past poses and the map are jointly refined using all available measurements. Within this category, graph-based SLAM has become particularly popular in the visual SLAM community due to improved consistency, reduced linearization

error, and greater robustness to loop closures compared to filtering methods.

Within visual SLAM, two main paradigms emerged: feature-based and direct methods. Feature-based methods rely on salient keypoints across images to establish geometric constraints between two viewpoints. PTAM [7] was the first method to split camera tracking and mapping in parallel threads using FAST corners [8] for feature matching. PTAM was influential for subsequent work as it demonstrated real-time monocular SLAM with parallel threads. However, its lack of loop closure handling, relocalization, and scalability restricted its use to small, controlled workspaces. ORB-SLAM [9] addressed these issues by introducing a feature-based monocular SLAM system that extends its capabilities to allow accurate keyframe selection, bundle adjustment, and loop-closure pose graph optimization. While feature-based methods are generally robust and interpretable, feature extraction can fail in low-texture or repetitive environments. On the contrary, direct methods bypass explicit feature extraction by directly minimizing photometric error over image pixel intensities, resulting in dense reconstructions. LSD-SLAM [10] was one of the first systems to demonstrate large-scale, real-time monocular SLAM with direct methods. It constructs a pose graph of keyframes, each containing an estimated semi-dense depth map using a novel direct image alignment formulation. DSO (Direct Sparse Odometry) [11] improved robustness and accuracy by formulating visual odometry as a photometric bundle adjustment over selected high-gradient pixels, enabling precise camera tracking with reduced computational overhead. SVO [12] offers a hybrid system that extracts FAST features, uses a direct method to track features, and achieves real-time performance on resource-constrained platforms such as micro aerial vehicles. Despite their advantages over feature-based methods, direct methods have significant limitations. These methods assume brightness constancy between frames, which is highly affected by dynamic objects, changing lighting conditions, and shadows. Furthermore, they require accurate photometric and geometric calibration for precise tracking performance.

In general, scalability, sensitivity to measurement noise, dependence on sensor calibration, long-term robustness, and lack of semantics remain as open challenges for metric methods. Since both filtering-based and graph-based formulations require maintaining and updating a large number of variables, this can become computa-

tionally expensive as the map grows. Metric maps need accurate measurements, and small inaccuracies in sensor calibration or measurement can accumulate into significant drift. They often work with static world assumptions and struggle to adapt to lighting variations and scene modifications over time. Finally, a critical limitation of purely metric approaches is the lack of semantic understanding required in high-level task planning.

2.2 Topological Mapping

In contrast to metric mapping, which estimates the precise geometry, topological mapping prioritizes place abstraction, connectivity, and qualitative spatial relations. The notion of encoding environment as linked places rather than dense geometry is inspired by cognitive mapping and qualitative spatial reasoning in humans and animals [13]. As a consequence, the place recognition and appearance-based methods are widely used to build and update large-scale maps without metric information. FAB-MAP [14] employs a bag-of-words image retrieval system to assign each new observation to either a new or previously visited location and incrementally builds a purely topological map representation in large-scale environments. However, FAB-MAP relies heavily on handcrafted visual features, and its performance degrades under drastic appearance changes. SeqSLAM [15] localizes by identifying coherent sequences of best-matching locations across navigation sequences, rather than relying on single-image matches. Although SeqSLAM claims to handle extreme perceptual changes, it relies on a major assumption about route similarity among sequences.

More recent works utilized topological representations with deep-learning-based methods for developing navigation strategies. SPTM [16] constructs a non-parametric memory graph and a parametric retrieval network, storing only connectivity and enabling navigation from just minutes of exploratory video in unseen mazes. PlaceNav [17] uses visual place recognition to select subgoals within a topological map and employs Bayesian filtering to ensure temporal consistency when choosing these subgoals. Active Neural SLAM blends analytical path planners with a learned SLAM module, global and local policies. Wiyatno [18] et al. propose a sampling-based topological graph construction and lifelong maintenance strategy that improves nav-

igation performance and adaptability. Learning based methods often face limitations in generalizability, requiring retraining or fine-tuning for new environments or generating policies that may not transfer well across diverse settings.

Topological mapping offers a scalable and robust way to represent environments by focusing on places and their connections rather than exact geometry. However, existing approaches still have notable shortcomings. The place discretization creates a tradeoff between scalability and accuracy. Reliable place recognition remains difficult due to issues like perceptual aliasing and changes in appearance, especially in dynamic indoor layouts. Graph maintenance remains vulnerable to data association errors over time. Critically, most approaches operate with little or no semantic understanding of objects and regions. This absence of semantics restricts reasoning about tasks, limits robustness in aliased environments, and prevents the maps from supporting long-term, task-driven autonomy.

2.3 Semantic SLAM

Semantic SLAM enhances traditional geometrical mapping by integrating the information of meaningful scene elements, such as objects or places. This integration supports not only localization and motion planning, but also scene understanding, reasoning, and task planning. An early seminal work, SLAM++ [19], demonstrates real-time incremental SLAM in large, cluttered environments, including loop closure, relocalisation, and the detection of moved objects. However, SLAM++ used a priori object CAD model database to construct 3D semantic maps, which limits its applicability to environments where the objects are unknown or not part of the predefined database.

Semantic mapping methods that combine real-time SLAM with deep learning-based segmentation methods have emerged. SemanticFusion [20] was one of the first systems that fused per-frame predictions into a dense semantic map. Following this approach, MaskFusion [21] integrated instance-level segmentation masks from Mask R-CNN with dense SLAM, enabling object tracking, both static and dynamic, and reconstructing a dense map from individual objects in a scene. PanopticFusion [22]

takes the next step by unifying semantic class labels for *stuff* (e.g., walls, floors) and instance-level representations for *thing* (e.g., chairs, tables), thereby generating a complete panoptic 3D map. These approaches inherit several common challenges. First, the combination of segmentation and dense 3D reconstruction leads to substantial computational overload, restricting scalability to larger or long-term deployments. Second, these methods lack strong mechanisms for ensuring global consistency over long trajectories, which can result in cumulative pose drift.

QuadricSLAM [23] uses a general parametric representation of objects as quadric surfaces to model the environment. This approach simplifies the map, providing a more compact and structured representation, and allows object poses and shapes to be optimized alongside camera trajectories in a factor graph.

More recently, Kimera [24] has provided a real-time framework that produces a 3D metric-semantic mesh from semantically labeled images. Kimera is a modular framework consisting of four key components: a visual-inertial odometry (VIO) module, a pose graph optimizer, a 3D mesher for fast mesh reconstruction, and a dense 3D metric-semantic reconstruction module for detailed environment mapping. Kimera’s performance relies heavily on the quality and accuracy of sensor data, with noise or errors potentially degrading map quality, and its assumption of static objects for mapping may limit its effectiveness in dynamic environments.

Beyond purely metric-semantic systems, recent research has explored topological formulations that abstract away from full metric consistency while leveraging semantic structure. Chaplot et al. [25] propose a supervised learning-based algorithm to construct topological map representations in which nodes are enriched with semantic features and connected using coarse geometric relations. RoboHop [26] introduces an alternative paradigm where nodes correspond not to images or pixels, but to semantically meaningful image segments that are open-vocabulary queryable.

In general, the performance of semantic SLAM algorithms is strongly dependent on the quality of segmentation and data association. Especially for purely metric approaches that tightly couple semantic and metric information, both geometric and semantic errors can propagate, leading to inconsistencies in mapping. Moreover, most approaches struggle with scalability, as object-based maps grow rapidly over

long trajectories, making retrieval and reasoning computationally expensive. Finally, the current approaches are vulnerable to dynamic environments where objects disappear or are moved over time. These limitations highlight a need for a complete, robust, scalable semantic SLAM solution that can maintain reliable performance in long-term deployments.

2.4 Hierarchical Map Representations

The development of hierarchical map representations emerged as a solution to the scalability problem. Traditional SLAM algorithms, particularly filtering-based ones, exhibit a computational cost that grows quadratically or even cubically with the size of the environment and the number of landmarks being tracked. Hierarchical representations organize spatial information at multiple levels of abstraction, decomposing the environment into a series of smaller groups or regions that can be independently managed. Seminal work by Estrada et al. [27] introduced a two-level decomposition, where the lower level is composed of a set of local maps that are statistically independent, and the upper level is an adjacency graph that captures relative spatial relationships between these local maps. In parallel, Bosse et al. [28] proposed a similar approach with the Atlas system, which represents the environment as a graph of limited-size local maps, or *charts*. The Atlas system is designed so that its computational cost scales with the number of sensor measurements, rather than the size of the mapped region, achieving constant time performance per iteration.

Hybrid models [29, 30] were proposed to combine the strengths of metric and topological approaches. In general, these methods use a global topological map that connects local metric submaps. Global topological map manages loop closures robustly and compactly, whereas the local metric submaps provide accurate pose estimation for fine-grained motion planning.

Specifically in visual SLAM, typically, an image is encoded as a vector of specified length by means of a global descriptor. A similarity metric appropriate to that type of descriptor is utilized for image comparison to find the closest match for a query image. Exhaustive search through all encoded images using this metric results in a

search time that increases linearly with the total number of images in the map. To overcome this, hierarchical representations of the environment have been proposed, where visually similar images are grouped into nodes to reduce the search space [31].

Semantic information is inherently hierarchical, making it a natural fit for multi-level mapping architectures. A scene can be represented as a multi-layered graph, often referred to as a *3D Scene Graph* [32]. Here, environments are represented at multiple resolutions: low-level geometric information for precise localization, object-level representations for semantic reasoning, and higher-level topological or place graphs for task-level planning.

Despite the significant progress, several challenges remain. Data association continues to be a key problem in complex and dynamic environments. Similarly, the development of generalizable and robust semantic classification is still an open problem.

CHAPTER 3

METHODOLOGY

In this chapter, we outline our approach to visual robot localization and topological mapping using spherical camera images. Our method integrates three main components: relative pose estimation, graph-based map construction, and object-based visual place recognition. To estimate the relative pose between frames, we extract features and match pixels, then compute the essential matrix according to the spherical camera model explained in detail. We apply RANSAC, to remove outliers and reduce errors in the correspondences.

Using the relative pose estimates, we build a graph-based topological map where each node represents a robot pose and edges capture spatial constraints between them. We employ pose graph optimization techniques to refine the graph and minimize cumulative errors, resulting in a globally consistent map. To enable loop closure detection and maintain reliable long-term localization, we introduce an object-based visual place recognition approach. This method uses panoptic segmentation and mask association to track persistent objects across frames, combining their features into global descriptors.

Overall, our methodology combines geometric, graph-based, and semantic information to create a comprehensive framework for reliable vSLAM with spherical images.

3.1 Relative Pose Estimation

Estimating the relative pose between two images taken from distinct viewpoints is a crucial step in robot localization and mapping. Given a pair of images captured

from two different poses, the goal is to determine the transformation that relates one camera center to the other. This estimation serves as a backbone for constructing the pose graph of the environment within the scope of this thesis.

The typical pipeline consists of extracting salient features from both images, finding pixel-level correspondences between these features, and using them to establish a geometric constraint. This is commonly known as the *epipolar constraint*, which relies on the calculation of the fundamental and essential matrices. Since spherical projection maps 3D points to vectors on a unit sphere instead of projecting them onto a plane, as in the case of perspective cameras, the essential matrix and fundamental matrix become equivalent under the spherical camera model assumption, as will be demonstrated mathematically later in this chapter. The relative pose of the second camera can be recovered from the essential matrix using Singular Value Decomposition (SVD).

Several methods have been proposed in the literature to derive the essential matrix from a set of pixel correspondences. The most widely used approaches are the 5-point algorithm and the 8-point algorithm (8-PA). In this thesis work, 8-PA coupled with RANSAC is used to robustly estimate the essential matrix by removing outlier correspondences.

3.1.1 Keypoint Feature Extraction

Identifying salient and repeatable features has been a fundamental challenge in computer vision, with a wide range of applications such as structure-from-motion, object recognition, image matching, and vSLAM. Such features need to be invariant under changes in illumination, scale, rotation, and viewpoint to be effective across these applications. After keypoint features are detected, they must be represented with a fixed-size vector that encodes the local information around the detected feature. These vectors, namely *descriptors*, can be used for matching features across frames. Since pose estimation performance is heavily dependent on consistently detecting and matching features, their repeatability is critical. In our application, the even distribution of features across the 360° FOV is especially important. The accumulation of keypoint features along a narrow direction can dominate RANSAC and result in

biased relative pose estimations. Along with these requirements, feature extraction must provide real-time performance for vSLAM algorithms to be deployed on mobile robots. Therefore, the selection of a feature extraction algorithm that finds a good balance between robustness, accuracy, and computational efficiency is essential for a precise pose estimation pipeline.

Hand-crafted feature detection and description methods such as SIFT, ORB, and BRISK have been widely used in a variety of computer vision applications. These methods generally make use of pixel intensity gradients and manually designed heuristics to identify and describe distinctive pixels. SIFT is a pioneering approach that provides accurate, scale, and rotation invariant features, yet its high computational overhead limits its suitability for real-time SLAM systems. On the other hand, ORB features provide binary descriptors that yield faster computation times; therefore, ORB is a core component of one of the leading approaches in vSLAM, ORB-SLAM, and its variants. However, these methods are specifically crafted for perspective images, and their performance on spherical images degrades due to strong distortions caused by equirectangular projection. The deformations are caused by non-uniform sampling of the sphere and are more dramatic around the poles. As 360° images become more popular, adaptations of these traditional methods to handle spherical distortions have been proposed [33, 34]. These methods work in spherical coordinates instead of Euclidean 2D images, account for projection distortions, and spherical neighborhood.

With the advancements in deep learning, feature detection and description methods such as SuperPoint [35] and LOFTR [36] have gained popularity. Instead of manually crafted heuristics, these feature extractors are trained on large datasets to detect and describe keypoints in an end-to-end process. They achieve state-of-the-art accuracy with a slightly higher computational cost compared to conventional methods.

3.1.2 Spherical Camera Model and Unit Vector Projection

A spherical camera model represents an image as a unit sphere centered at the camera origin. Under this model, a world point $\mathbf{X} \in \mathbb{R}^3$ is projected onto a unit sphere centered at the camera position $\mathbf{C} \in \mathbb{R}^3$ resulting in a unit vector $\mathbf{x} \in \mathbb{S}^2 \subset \mathbb{R}^3$ that points from the camera center to the 3D point $\mathbf{X}$. Figure 3.1 illustrates the projection

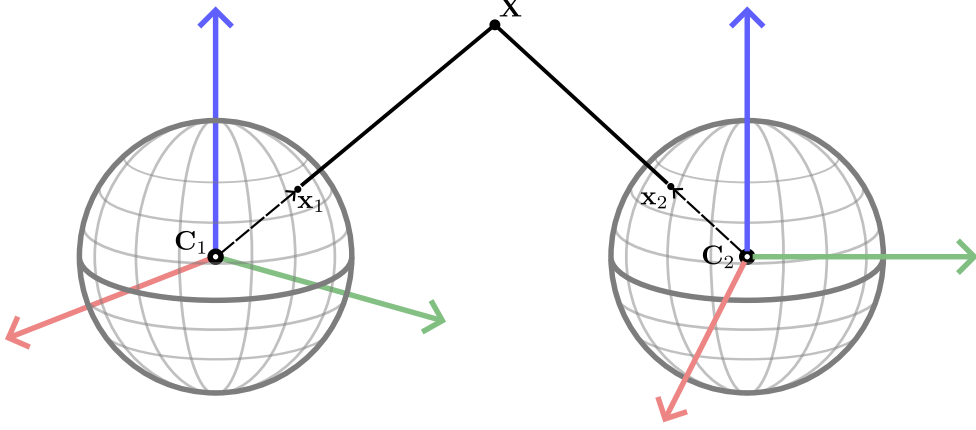


Figure 3.1: Projection of a world point onto two spherical cameras.

of a world point $\mathbf{X}$ to spherical cameras at two different locations. The mapping from 3D world coordinates to the sphere can be written as.

$$\mathbf{x} = \frac{\mathbf{X} - \mathbf{C}}{\|\mathbf{X} - \mathbf{C}\|}. \quad (3.1)$$

Unlike perspective cameras, where projection is modeled as $\mathbf{x} = K [R \mid t] \mathbf{X}$, the spherical model does not include intrinsic parameters such as focal length or principal point. Therefore, camera extrinsics are sufficient to describe its relationship with the world. The camera extrinsics $[R \mid t]$ consist of a rotation matrix $R \in SO(3)$ and a translation vector $t = -R \mathbf{C} \in \mathbb{R}^3$.

Let $\mathbf{x}_1$ and $\mathbf{x}_2$ be the projections of the same 3D point $\mathbf{X}$ on two distinct spherical cameras. Since spherical cameras lack intrinsic parameters, without loss of generality, the epipolar geometry defined for perspective cameras describes a geometrical constraint between the pair of spherical images:

$$\mathbf{x}_2^\top \mathbf{E} \mathbf{x}_1 = 0, \quad \mathbf{E} = [\mathbf{t}]_\times R, \quad (3.2)$$

where $\mathbf{E} \in \mathbb{R}^{3 \times 3}$ is the essential matrix, $[\mathbf{t}]_\times$ stands for the skew-symmetric matrix associated with the cross-product with $\mathbf{t}$.

Although the spherical camera model explained so far provides a convenient mathematical formulation, there is no single sensor that can capture light from all directions. Spherical images are typically represented in the equirectangular format. Equirectan-

gular projection maps 3D spherical surface to a 2D rectangle, where pixel coordinates correspond to the azimuth angle $\theta \in [-\pi, \pi]$, and the elevation angle $\phi \in [-\frac{\pi}{2}, \frac{\pi}{2}]$. Let an image have width W and height H . A pixel at image coordinates (u, v) , the conversion to angular values can be expressed as:

$$\theta = 2\pi \frac{u}{W} - \pi, \quad (3.3)$$

$$\phi = \frac{\pi}{2} - \pi \frac{v}{H}. \quad (3.4)$$

Given (θ, ϕ) , we compute the unit vector $\mathbf{x} \in \mathbb{S}^2$ as:

$$\mathbf{x} = \begin{bmatrix} \cos \phi \cos \theta \\ \cos \phi \sin \theta \\ \sin \phi \end{bmatrix}. \quad (3.5)$$

This vector represents the direction of the light ray from the camera center through the image pixel (u, v) and is used directly in the essential matrix calculation.

3.1.3 Essential Matrix Estimation and Pose Recovery

Given a set of point correspondences $\{\mathbf{x}_1^i \leftrightarrow \mathbf{x}_2^i\}_{i=1}^N$, $N \geq 8$, where $\mathbf{x}_1^i, \mathbf{x}_2^i \in \mathbb{S}^2$ are unit vectors obtained through the feature extraction and matching pipeline in the first and second frames, respectively, well-known 8-PA algorithm can be directly applied for estimating the essential matrix $\mathbf{E}$.

By rearranging the terms, Equation 3.2 can be reformulated into a homogeneous system of linear equations as follows:

$$\mathbf{A} \mathbf{e} = \mathbf{0}, \quad (3.6)$$

where $\mathbf{e}$ is the 9×1 vector of the elements in matrix $\mathbf{E}$ concatenated row-wise, and $\mathbf{A}$ is an $N \times 9$ matrix, with each row constructed from a pair of point correspondences. Specifically, the i -th row of $\mathbf{A}$ is the Kronecker product of $\mathbf{x}_1^i$ and $\mathbf{x}_2^i$:

$$\mathbf{a}_i = \begin{bmatrix} x_2^i x_1^i & x_2^i y_1^i & x_2^i z_1^i & y_2^i x_1^i & y_2^i y_1^i & y_2^i z_1^i & z_2^i x_1^i & z_2^i y_1^i & z_2^i z_1^i \end{bmatrix}. \quad (3.7)$$

We apply SVD to $\mathbf{A}$ and find the least squares solution to the Equation 3.6.

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top \quad (3.8)$$

The non-trivial solution for $\mathbf{e}$ is given by the last column of the orthogonal matrix $\mathbf{V}$. Alternatively, it can be described as the right singular vector corresponding to its smallest singular value of the matrix $\mathbf{A}$. This vector can then be reshaped into a 3×3 initial estimate of essential matrix $\mathbf{E}$. However, the matrix estimated from the solution does not perfectly agree with the constraints of an essential matrix due to noise in the point correspondences. By definition, an essential matrix has to have rank 2. Therefore, the SVD of the estimated matrix is computed and reconstructed by setting the lowest singular value to zero, enforcing a rank 2 constraint. This step can be interpreted as the projection of the estimated essential matrix onto the space of valid essential matrices.

$$\mathbf{E} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top \quad (3.9)$$

$$\mathbf{E}' = \mathbf{U} \text{diag}(\sigma_1, \sigma_2, 0) \mathbf{V}^\top \quad (3.10)$$

After the essential matrix $\mathbf{E}'$ is reconstructed, 3 DoF of the rotation matrix $\mathbf{R}$ and 2 DoF of the translation vector $\mathbf{t}$ between two camera frames can be recovered. The projection of the 3D world points during image formation results in scale ambiguity in monocular vision. Therefore, the translation vector is only known up to a scale without any additional metric information. The topological framework used in this work eliminates the dependence on metric scale knowledge. The relative heading information between two camera poses is sufficient to construct a topological graph. Since navigation in topological maps reduces to traversal over neighboring nodes, the absolute scale information is not necessary. The essential matrix yields four possible motion hypotheses for the relative pose. Let $\mathbf{E}' = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$ be its SVD; and define

$$\mathbf{W} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The following candidate solutions for rotation and translation can be derived.

$$R_1 = \mathbf{U} \mathbf{W} \mathbf{V}^\top, \quad \mathbf{t}_1 = \mathbf{u}_3, \quad (3.11)$$

$$R_2 = \mathbf{U} \mathbf{W}^\top \mathbf{V}^\top, \quad \mathbf{t}_2 = \mathbf{u}_3, \quad (3.12)$$

$$R_3 = \mathbf{U} \mathbf{W} \mathbf{V}^\top, \quad \mathbf{t}_3 = -\mathbf{u}_3, \quad (3.13)$$

$$R_4 = \mathbf{U} \mathbf{W}^\top \mathbf{V}^\top, \quad \mathbf{t}_4 = -\mathbf{u}_3, \quad (3.14)$$

where $\mathbf{u}_3$ is the third column of $\mathbf{U}$. The correct solution among these four candidates is typically determined using the cheirality condition, which enforces that triangulated 3D points lie in front of the cameras. This positive depth constraint is applicable for perspective cameras. However, in the case of the spherical cameras, the term “in front of the camera” becomes irrelevant due to the 360° FOV. Instead, we select the motion hypothesis that minimizes the angular projection error between observed image rays and their projected directions for each corresponding candidate camera pose.

3.1.4 Robust Estimation using RANSAC

In theory, having at least eight feature correspondences is sufficient to estimate the relative pose between two spherical cameras. However, in practice, feature correspondences often contain outliers due to sensor noise, image distortions, and errors in the feature matching pipeline. The presence of these outliers hinders the accurate estimation of the essential matrix. Therefore, a method for handling outliers is required for a precise pose estimation between two viewpoints. In this work, the RANSAC algorithm is used to reject incorrect feature correspondences and calculate an essential matrix in a robust manner.

RANSAC is an iterative method that fits a model based on a randomly sampled subset of the data and searches for the best model parameters that result in the largest consensus. In our case, the RANSAC problem can be rephrased as finding a subset of eight point correspondences for estimating the essential matrix that agrees with the largest number of matched features.

At each iteration, the algorithm randomly selects a subset of eight point correspondences. A candidate essential matrix is estimated using 8-PA, as described in Section

3.1.3. Then, for all feature pairs, a projection error is calculated using the epipolar constraint given in Equation 3.3. Pairs with errors less than a preset threshold are considered as inliers. RANSAC tracks the essential matrix model with the highest number of inliers through all iterations. The algorithm is repeated until the iteration count reaches a dynamically calculated maximum iteration threshold, based on confidence. At the end, the final essential matrix is estimated using all the inlier feature pairs from the best-fitting model. This process considerably improves the accuracy of the pose estimation by rejecting mismatched point correspondences.

3.2 Graph-Based Construction of Topological Map

Graph-based representations offer an intuitive way of formulating the SLAM problem that prioritizes connectivity and spatial relations over metric accuracy. Although topological maps are less sensitive to precision loss, accumulated errors in relative pose estimation can still lead to inconsistencies in the topology, such as misaligned loops and drifted locations. Hence, it is still necessary to build a spatially consistent map that is not necessarily metrically accurate. In a topological map, an environment is modeled as a graph whose nodes represent distinct robot poses, and in which an edge encodes a spatial constraint between connected nodes that comes from observations. Since observations are subjected to noise, the edge constraints are usually contradictory. This creates a need for calculating the configuration of robot poses that is maximally consistent with these constraints. The graph-based SLAM is usually divided into two main tasks, namely the front-end and the back-end. The front-end refers to the process of incrementally constructing a graph by estimating constraints based on the observations. Once such a graph is constructed, the back-end refines pose estimates by solving a least-squares optimization problem [37]. The resulting map dictates node poses that satisfy multiple constraints with minimal error. This approach is an alternative to filtering-based methods widely used in SLAM, and it is a key concept in the advancement of vSLAM algorithms.

In our work, we use a graph-based framework where each node corresponds to a discrete spherical camera position connected by edges that represent spatial constraints. The front-end, described in detail in Section 3.1, detects visual correspon-

dences between spherical images and estimates relative transformations using 8-PA with RANSAC. The back-end refines the initial camera pose estimates by performing pose graph optimization.

3.2.1 Graph Structure

In this section, we formulate a 2D pose graph optimization problem to obtain a maximum likelihood estimate of all robot poses. Let the graph representing the topological map be defined as

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}) \quad (3.15)$$

Here, $\mathcal{V}$ is the set of nodes corresponding to keyframes captured by a spherical camera and $\mathcal{E}$ is the set of edges representing the pairwise relative transformation obtained from visual odometry. Since mobile robot motion is generally assumed to be planar, the pose representation reduces to a 2D translation and orientation. Each node stores a pose $\mathbf{x}_i \in SE(2)$ that is represented as

$$\mathbf{x}_i = \begin{bmatrix} x_i & y_i & \theta_i \end{bmatrix}^\top \quad (3.16)$$

where (x_i, y_i) is the 2D position and $\theta_i \in (-\pi, \pi]$ is the yaw angle.

Each edge $(i, j) \in \mathcal{E}$ stores a relative pose between node i and node j . Relative pose measurement is obtained from the estimated essential matrix provided by the front-end, and decomposed into a 2D relative translation, $\mathbf{t}_{ij}$, and a relative yaw angle θ_{ij} . The actual measurement, $\mathbf{z}_{ij}$, is expressed as

$$\mathbf{t}_{ij} = \begin{bmatrix} t_x \\ t_y \end{bmatrix}, \quad \phi_{ij} = \arctan 2(t_y, t_x) \quad (3.17)$$

$$\mathbf{z}_{ij} = (\phi_{ij}, \theta_{ij}) \quad (3.18)$$

Where ϕ_{ij} is the heading angle that points from node i towards node j . Since these measurements lack absolute scale information, the translation is assumed to be unit length before optimization.

The iterative refinement process of the optimization requires a good initial estimate of robot poses. Otherwise, solvers may get stuck at local minima or take too long to

converge to a solution. During the initialization of the pose graph, the starting frame is selected as the anchor node with $\mathbf{x}_0 = (0, 0, 0)$. Graph nodes are initialized using breadth-first traversal starting from the anchor node. For every added edge (c, n) with current node c and neighboring node n , the initial estimate is propagated as given as follows.

$$\mathbf{R}(\theta_c) = \begin{bmatrix} \cos \theta_c & -\sin \theta_c \\ \sin \theta_c & \cos \theta_c \end{bmatrix}, \quad (3.19)$$

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} x_c \\ y_c \end{bmatrix} + \mathbf{R}(\theta_c) \mathbf{t}_{cn}, \quad (3.20)$$

$$\theta_n = \theta_c + \theta_{cn} \quad (3.21)$$

The initialization process generates a graph based on relative measurements, but it is not optimal due to contradictory constraints from multiple neighbors. After initializing the graph for a set of nodes $\mathcal{V} = \{\mathbf{x}_0, \dots, \mathbf{x}_n\}$ where $\mathbf{x}_i$ describes the pose of node i , an error is calculated as the difference between the expected measurement for the given graph configuration and the actual measurement captured by the spherical camera. Introduced notation for pose graph optimization is illustrated in Figure 3.2.

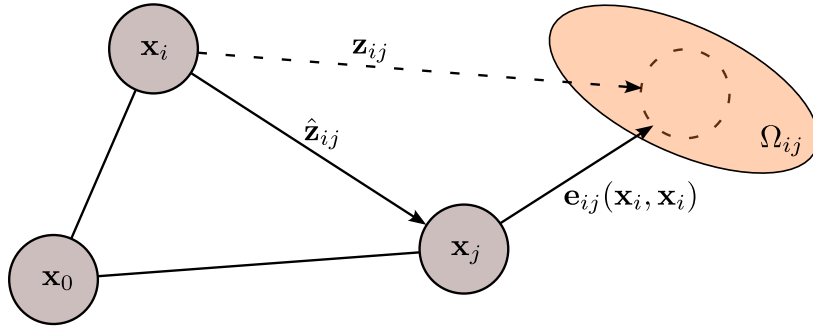


Figure 3.2: The notation for pose graph optimization illustrated on an arbitrary graph.

$$\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{z}_{ij} - \hat{\mathbf{z}}_{ij}(\mathbf{x}_i, \mathbf{x}_j) \quad (3.22)$$

where the expected measurement, $\hat{\mathbf{z}}_{ij}(\mathbf{x}_i, \mathbf{x}_j)$, is defined as the relative transformation between the poses of nodes $\mathbf{x}_i$ and $\mathbf{x}_j$ given the current map configuration.

$$\hat{\mathbf{z}}_{ij} = (\hat{\phi}_{ij}, \hat{\theta}_{ij}) \quad (3.23)$$

$$\hat{\phi}_{ij} = \arctan 2(y_j - y_i, x_j - x_i) \quad (3.24)$$

$$\hat{\theta}_{ij} = \theta_j - \theta_i \quad (3.25)$$

$$\mathbf{e}_{ij} = \begin{bmatrix} \phi_{ij} - \hat{\phi}_{ij} \\ \theta_{ij} - \hat{\theta}_{ij} \end{bmatrix} \quad (3.26)$$

The goal of this approach is to find a configuration for the set of nodes $\mathcal{V}$ that minimizes the cumulative error of all observations. In our setup, the robot moves on a planar surface, so the estimated pitch and roll angles should ideally be zero. We exploit this assumption to quantify the uncertainties in the visual odometry measurements. Through an experimental study, we observed a linear relationship between off-plane rotation and heading error. Using this relationship, we constructed a form of information matrix. Incorporating the information matrix ensures that more reliable measurements contribute more significantly to the resulting optimization. The optimal configuration is obtained by minimizing the sum of weighted error terms across all edges in the graph.

$$\mathbf{x}^* = \arg \min_{\mathbf{x}_i} \sum_{(i,j) \in \mathcal{E}} \mathbf{e}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij} \quad (3.27)$$

The variables to be optimized are the three pose parameters. In practice, we use an implementation of the Levenberg–Marquardt algorithm to solve the nonlinear least-squares optimization problem. At each iteration, the algorithm updates the pose estimates, computes the error, and calculates the Jacobian to approximate how the error changes with the pose parameters.

3.3 Visual Place Recognition and Loop Closure

Map building in graph-based representations requires finding image pairs that are spatially close to each other. These image pairs are treated as neighboring nodes and connected by an edge to express traversability between two robot poses. Finding a correct and complete set of neighboring nodes is essential for the accuracy of pose

graph optimization and path planning during navigation. The most straightforward approach is to compare each node to every other node in the graph and check for visual similarity. A simple method is to perform keypoint feature matching for each pair and assess visual similarity based on the number of matching keypoints. Although the per-pair comparison cost depends on the number of keypoints and the selected matching method, due to the pairwise structure, search time increases quadratically with the number of nodes in the graph. Moreover, this approach results in a large number of redundant comparisons between nodes that are far from each other. As a result, exhaustive search across all graph nodes is not a feasible method for real-time applications and computationally constrained systems.

A common strategy used in the literature to reduce unnecessary comparisons is to apply a fixed-size temporal window, comparing the current frame only with the most recent frames. This short-term association assumes incremental robot motion between consecutive frames and is prone to error accumulation over time. On the other hand, it significantly reduces the number of images to be evaluated and enables locally consistent maps with real-time performance. Solely relying on the last few frames fails to capture associations that are spatially proximate but temporally distant, leading to global inconsistencies and missed loop closures. To connect non-consecutive neighboring nodes, the current observation can first be localized based on the recent frames, and the spatial information can be used to find candidate nodes within a distance range. Integrating temporal and spatial proximity significantly improves mapping efficiency by avoiding pairwise comparisons between distant nodes without sacrificing much of the accuracy. A major flaw in the described approach is the detection of revisited places in the presence of a high accumulation of error. For example, in a scenario where the robot traverses a long corridor loop, as commonly found in office buildings, drift tends to accumulate as a result of the robot moving in a long, straight trajectory with only incremental measurements. Once the robot loops back to its starting point, previously visited nodes may fall outside the range due to accumulated error, preventing loop closure detection.

A complete vSLAM algorithm requires long-term data associations to detect loop closures beyond immediate neighbors. Numerous solutions can be adopted from visual place recognition literature, such as DBoW2 [38], NetVLAD [39], or more recent

learning-based methods [40, 41, 42]. In general, these methods work by creating a compact descriptor for images that enables efficient comparison between a query image descriptor with a database of known-place descriptors and returns the most visually similar observations. The compact descriptors enable a robot to recognize places it has previously seen in large-scale environments. This capability is valuable for both performing loop closures to correct accumulated drift during mapping and for localization on a constructed map.

VPR methods rely solely on appearance-based associations and do not consider spatial proximity or geometric constraints during retrieval. To integrate into a vSLAM pipeline, a geometric validation step is necessary to verify the consistency of candidate matches.

In this thesis, we implement a temporal window for matching immediate neighbors and localize the current node on the map based on the relations with immediate neighbors. Based on this initial estimate, spatially proximate nodes within a certain distance range are considered as neighbor candidates. Finally, we propose a novel VPR method that exploits the surrounding objects in the environment to perform robust place recognition. The proposed methods rely on finding segment-level associations between frames to obtain a global descriptor for each distinctive object. The object representations are utilized as landmarks to identify long-term data associations and match previously visited places based on covisible objects between frames. Note that these *landmarks* are not predefined or specially structured objects but consist of everyday items that can be commonly found in indoor environments. Using objects for place recognition has the following advantages:

- In this approach, we search for objects instead of images, as opposed to other VPR methods. Since the number of distinctive objects is smaller than the number of images in a densely mapped environment, the search complexity is significantly reduced.
- Previous VPR methods encode the entire frame into a single descriptor. As a result of representing the whole image with a single vector, they may fail to capture the partial visual similarities between frames. This is especially important for SLAM implementation, as the viewpoint may change significantly

between two nearby frames. Our method generates a separate representation for each object, focusing on visually and semantically salient elements of the scene.

- Integrating semantic information and object-level representation into VPR improves robustness against vanishing or dynamic objects. The system can maintain reliable matches even if some objects move or vanish, as long as static objects constitute the majority of the current frame. The objects with a highly dynamic nature, like humans or office chairs, are explicitly disregarded, ensuring a stable place recognition.
- When more than three landmarks with known correspondences are observed in a 360° image, we compute bearing vectors for each object centroid. By comparing pairwise angles between bearing vectors, we can define a metric for disparity between two frames required for geometric validation of the candidate neighbors. Since we are comparing angles between directions, this metric does not require world-frame alignment.

3.3.1 Segmentation

The proposed VPR method requires capturing persistent elements that can serve as reliable anchors for long-term localization. To effectively, consistently, and accurately recognize the same objects, we need a generalizable yet compact representation for each distinctive object. This process can be divided into three subtasks: identifying and describing objects in each frame, finding correspondences between instances of objects across frames, and aggregating the descriptors for all instances of each object to obtain a global representation.

The first step is to partition images into regions corresponding to object classes such as table, sofa, desk, or walls. Although object detection models like YOLO [43] and Faster R-CNN [44] are shown to be successful in locating objects, the coarse bounding boxes limit the precision and lack the granularity necessary for scene understanding in cluttered indoor environments.

Semantic segmentation simply assigns a class label to each pixel in the image. In this

context, multiple instances of the same class are not distinguished, and all objects are treated as *stuff*, referring to uncountable and amorphous regions. For example, in a scene containing three chairs, semantic segmentation would label all chair pixels as *chair* without differentiating the pixels of each specific chair. Semantic segmentation is commonly used in autonomous driving applications, where distinguishing between broad scene elements such as road or sidewalk is sufficient.

In contrast, instance segmentation addresses the limitation of semantic segmentation by providing separate instance IDs for each object within *thing* class, representing countable objects and distinct entities. Returning to the chair example, instance segmentation would not just classify chairs but also distinguish them as *chair 1*, *chair 2*, *chair 3*. However, instance segmentation ignores *stuff* classes and does not generate a mask for uncountable objects. Knowing the boundaries of such objects may be critical for our indoor navigation task. For instance, we can infer free space by detecting the boundaries of the floor.

Panoptic segmentation aims to unify the strengths of both methods, assigning a class label and an instance ID to each pixel. In this approach, *thing* classes are both assigned a semantic label and a unique instance ID for distinguishing multiple instances of the same object. On the other hand, *stuff* classes are assigned only a semantic label since they refer to uncountable, amorphous objects. Panoptic segmentation is particularly suitable for our indoor mapping and navigation task, as it provides a complete labeling of the scene.

3.3.2 Mask Association

Once the individual objects have been segmented and identified, the next challenge is to find correspondences of the same objects across multiple frames. To achieve this, we make use of the extracted keypoint features from within each object's segmentation masks. This provides a set of feature descriptors that is confined to the object's boundary, and encodes the distinctive visual appearance patterns of it. This set of descriptors constitutes a *feature collection* that serves as a compact yet discriminative representation of the object in that specific frame.

More formally, let $\mathcal{O} = \{o_1, o_2, \dots, o_M\}$ denote the set of all segmented object instances across a set of frames, where each o_m corresponds to an object instance observed in a specific frame. For each instance o_m , we extract a set of local keypoint descriptors from within its segmentation mask:

$$\mathbf{F}_m = \{\mathbf{f}_{m,1}, \mathbf{f}_{m,2}, \dots, \mathbf{f}_{m,K_m}\}, \quad \mathbf{f}_{m,k} \in \mathbb{R}^d$$

where K_m is the number of detected keypoints for object o_m and d is the descriptor dimensionality, $d = 256$ for SuperPoint. We refer to $\mathbf{F}_m$ as a *local feature collection* for o_m .

We then seek to match the newly observed local feature collections with the previously stored feature collections. We first apply a soft-assignment of feature collections that evaluates their similarity. In this thesis work, we investigate two methods for calculating a similarity matrix:

1. Subspace distance-based similarity

In this approach, each feature collection is treated as a subspace spanned by its descriptors. In contrast to pair-wise descriptor matching, which treats each feature vector independently, we model the set of descriptors associated with a physical object as a low-dimensional subspace in the descriptor space. This formulation assumes that the descriptors belonging to the same physical object lie approximately on a linear subspace. Furthermore, the variation in descriptors for that object is assumed to be captured by the linear combination of a small set of basis descriptors. This subspace captures the essential visual patterns of the object while suppressing noise and redundant detail.

Given two feature collections $\mathbf{F}_i$ and $\mathbf{F}_j$, we compute a similarity score S_{ij} . Each feature collection is first centered to have zero mean, ensuring that similarity is not biased by global descriptor offsets. We represent each set $\mathbf{F}_m$ as a matrix $\mathbf{X}_m \in \mathbb{R}^{d \times K_m}$ where each column corresponds to a descriptor. We then compute orthonormal bases for each set using SVD.

$$\mathbf{X}_m = \mathbf{U}_m \mathbf{\Sigma}_m \mathbf{V}_m^\top, \quad \mathbf{U}_m \in \mathbb{R}^{d \times r} \quad (3.28)$$

where $r \leq \mathbf{K}_m$ is the subspace dimensionality that is sufficient to capture the essential visual patterns to describe the object.

Principal angles between the two subspaces are the angles that measure how "far apart" these two subspaces are. We define principal angle distance between two sets $\mathbf{F}_i$ and $\mathbf{F}_j$ as:

$$D_{\text{sub}}(\mathbf{F}_i, \mathbf{F}_j) = \left(\sum_{p=1}^r \theta_p^2 \right)^{1/2} \quad (3.29)$$

where θ_p are the *principal angles* obtained from the singular values of the matrix $\mathbf{U}_i^\top \mathbf{U}_j$.

2. Optimal transport-based similarity

While the subspace-based approach assumes that descriptors associated with a physical object lie approximately on a linear subspace, this assumption may not hold if the descriptors lie on a nonlinear manifold rather than a strictly linear subspace. We adopt an Optimal Transport (OT)-based similarity measure, which compares two sets of descriptors as probability distributions to handle if our data forms a more complex structure.

In this approach, each feature collection $\mathbf{F}_i$ is treated as a distribution and the OT framework seeks the most cost-effective way to "transport" the mass of one distribution to match the other. First, descriptors are normalized, and the pair-wise cosine distances between all descriptors in the two sets are computed to form the cost matrix $\mathbf{C}$. Uniform weights are assigned to each descriptor, reflecting equal importance. Then we compute a transport plan $\mathbf{P}$ that minimizes the total transport cost using the Sinkhorn algorithm.

$$\mathbf{P} = \arg \min_{\mathbf{P} \in \mathcal{U}(\mathbf{a}, \mathbf{b})} \sum_{k,l} C_{kl} P_{kl} \quad (3.30)$$

To derive a similarity score from the transport plan, we sum the transport mass corresponding to low-cost assignments. The low-cost metric highlights descriptors that are closely aligned across the two feature collections, while minimizing the influence of high-cost matches that often result from viewpoint differ-

ences or object variations.

$$S_{\text{OT}}(\mathbf{F}_i, \mathbf{F}_j) = \sum_{k,l} P_{kl} \cdot \mathbf{1}_{\{C_{kl} < \tau\}}, \quad (3.31)$$

where τ is a threshold that determines which transport pairs contribute to the similarity.

The output of the chosen soft-assignment method is a similarity matrix, where each entry encodes the similarity between two feature collections. We then apply a hard-assignment step to identify confident mask associations. For each row in the similarity matrix, we compute the z-score of all similarity values in that row and select entries whose z-score exceeds a predefined threshold, indicating statistically significant similarity.

$$z_{ij} = \frac{S_{ij} - \mu_i}{\sigma_i}, \quad \mu_i = \frac{1}{M} \sum_{j=1}^M S_{ij}, \quad \sigma_i = \sqrt{\frac{1}{M} \sum_{j=1}^M (S_{ij} - \mu_i)^2} \quad (3.32)$$

where μ_i and σ_i are the mean and standard deviation of similarity scores of the i_{th} row. Pairs with $z_{ij} > \tau$ are considered confident mask associations. These confirmed matches are represented as edges in a graph whose nodes correspond to individual object instances from specific frames. Repeating the process for all rows yields a graph that decomposes into disconnected subgraphs, where each subgraph represents all observations of the same physical object. This structure allows us to aggregate the per-frame feature collections within each subgraph into a global, compact representation for long-term object recognition and re-localization.

The mask association pipeline is summarized in Figure 3.3. Starting with local panoptic segmentation masks, one can observe that local object IDs change between frames. Using keypoint feature descriptors, a pairwise similarity matrix between all mask descriptors is then computed using the chosen similarity metric. To identify likely correspondences for a given mask, we examine the z-score distribution of its similarity values across all candidates, and selecting that exceptionally similar masks. Finally, we construct an undirected graph in which nodes correspond to mask instances and edges represent high-confidence associations. The connected components of this

graph correspond to unique physical objects observed across frames. This process allows robust cross-frame instance matching without requiring temporal continuity or motion tracking.

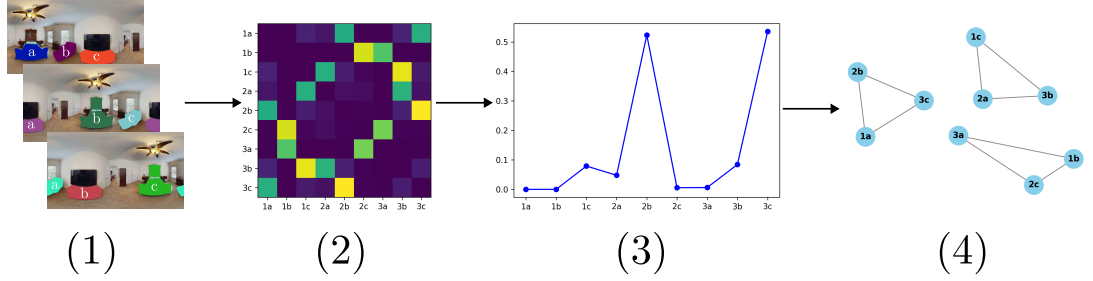


Figure 3.3: Mask association pipeline. **(1) Per-frame masks:** Panoptic segmentation produces object masks in each image; for every mask we extract a compact set of key-point descriptors restricted to the mask region. **(2) Similarity matrix:** We compute pairwise similarities between all mask-level descriptor sets using the chosen metric, forming a similarity matrix. **(3) Z-score selection:** For a query mask (e.g., $1a$), we convert its similarity row to z -scores and select entries whose similarity scores are statistical outliers on the high side as candidate matches. **(4) Association graph:** Masks are nodes and high-confidence matches become edges; connected components correspond to unique physical objects observed across frames.

3.3.3 Global Object Representation Aggregation

Once mask associations are established, all observations of the same physical object are grouped into a single connected component in the association graph. Each connected component corresponds to a unique physical object that is observed in multiple frames, from different viewpoints, and under varying illumination. The aggregation step aims to merge these *feature collections* from multiple frames into a single *global descriptor*. This global descriptor should capture the consistent and unique features of the object to create a representation that is discriminative enough to recognize the same object if it appears again later in the sequence.

The aggregation process depends on the similarity computation method used during mask association:

- **Subspace-based aggregation:** If we proceed under the assumption that the descriptors of a physical object lie approximately on a linear subspace in the descriptor space, we treat the complete set of descriptors as samples drawn from that subspace. We concatenate all descriptors from the associated object instances into a single matrix. A lower-rank approximation of this matrix is computed using SVD to get a small set of basis vectors. The resulting orthonormal basis $\mathbf{U}_r \in \mathbb{R}^{d \times r}$ forms a global representation for the object. When new feature collections are observed, principal angles or projection distance can be used to evaluate their similarity to existing subspaces. The global descriptor can be incrementally updated as more observations become available.
- **Optimal Transport-based aggregation:** In the OT-based method, the transport plan already provides a soft assignment between descriptors from different observations of the same object. This correspondence information enables us to avoid redundant storage of descriptors that represent the same physical points on the object’s surface. The aggregated representation is thus maintained as a set of unique descriptors, where each element corresponds to a distinct visual feature. This set can grow as new, non-redundant features are observed from novel viewpoints, ensuring comprehensive coverage of the object’s appearance.

Regardless of the aggregation strategy, all global object descriptors are stored in an indexed database. Each entry is keyed by a unique *object ID* and contains:

1. **Descriptor representation:** either the orthonormal basis vectors (subspace method) or the set of unique descriptors (OT method).
2. **Semantic class label:** the output of the panoptic segmentation model.
3. **Observation metadata:** a list of all nodes (frames) in which the object was observed, along with the bearing angles to the object centroid from each node. This information is later used for geometric verification.

3.3.4 Object-Based Place Recognition Query

After the global object database has been constructed, it can be queried to detect previously visited places based on object-level correspondences rather than full image matching. This stage aims to identify loop closure candidates by leveraging the distinctive and semantically meaningful global descriptors accumulated during the aggregation step.

Given a new query image that contains n detected objects, the proposed VPR algorithm attempts to find the most similar map node from the existing pose graph that is likely to be a nearby location.

For each detected object in the query image, its per-frame feature collection is compared to every global object descriptor in the database using the same similarity metric employed during mask association. This process yields a similarity matrix, $S^{n \times m}$, where m is the total number of objects in the database.

Each object in the query image is assigned to the map object with the highest similarity score, if that score is above a threshold, τ_{assign} . This step discards low-confidence object matches and considers only strong object correspondences. Each node gets a score based on the percentage of co-visible objects with confirmed matches. Candidate map nodes are selected from those sharing the greatest number of matched objects with the query image. Irrelevant nodes that do not share any objects are ignored without performing geometric verification.

3.3.5 Geometric Consistency Check

For candidate nodes with 3 or more shared objects, 2D bearing angles of the matched object centroids in both the query and candidate frames are retrieved for a geometric consistency check. We compute disparity by comparing pairwise angular distances between corresponding bearings in two observations, rather than the absolute orientations of the bearings. Let $\{\mathbf{b}_i^q\}_{i=1}^n$ and $\{\mathbf{b}_i^m\}_{i=1}^n$ denote the unit bearing vectors of the n matched objects in the query and map nodes, respectively. For each pair of objects (i, j) , we compute the angular distance between bearings in each node:

$$d_{ij}^q = \arccos \left(\frac{\mathbf{b}_i^q \cdot \mathbf{b}_j^q}{\|\mathbf{b}_i^q\| \|\mathbf{b}_j^q\|} \right), \quad d_{ij}^m = \arccos \left(\frac{\mathbf{b}_i^m \cdot \mathbf{b}_j^m}{\|\mathbf{b}_i^m\| \|\mathbf{b}_j^m\|} \right),$$

and define the pairwise angular disparity as

$$\delta_{ij} = |d_{ij}^q - d_{ij}^m|.$$

The overall node disparity is obtained by averaging over all unique pairs:

$$\Delta = \frac{2}{N(N-1)} \sum_{i < j} \delta_{ij}.$$

This method has the advantage of not requiring a global orientation: since only relative angles are compared, a pure yaw rotation (e.g., a robot turning in place) shifts all bearings equally, leaving the pairwise angular differences unchanged. However, it is sensitive to translation, which changes the observed pairwise angles. The approach is also computationally efficient as it involves only simple trigonometric operations such as dot products and arccos.

We rank candidates by the disparity score and keep the top- k nodes as loop-closure hypotheses. Each hypothesis is then confirmed by sparse feature matching between the query image and the candidate node’s images. This sparse feature matching is also utilized by the relative pose estimation with 8-PA. Finally, confirmed matches produce loop-closure constraints that are inserted into the pose graph.

To conclude, the methodology presented in this chapter outlines a robust approach to topological mapping and localization using spherical cameras. By combining relative pose estimation, graph-based optimization, and object-level visual place recognition, the system achieves both local mapping accuracy and global consistency in complex environments. Through the use of panoptic segmentation and mask-based object tracking, it becomes possible to detect loop closures without relying heavily on full-frame image descriptors. Geometric verification based on object bearing vectors further improves reliability, especially under viewpoint changes and in dynamic scenes. By decoupling place recognition from full-frame matching and operating on a smaller set of distinctive, semantically meaningful entities, the search space is dras-

tically reduced. Together, these components form a scalable and efficient vSLAM pipeline by integrating geometric cues, semantic understanding, and graph structure, the system provides a solid foundation for long-term autonomous navigation in real-world settings.

CHAPTER 4

IMPLEMENTATION

In this chapter, we present the implementation details of the proposed algorithms, beginning with the mapping procedure. Building on this, we introduce two navigation strategies that operate on the constructed topological map. The first navigation method is developed for 360-degree FOV images; it is designed for a differential drive robot equipped with a spherical camera. The second algorithm also uses the graph built from spherical images, but is applied to a robot with a perspective camera of an ambiguous field of view. This demonstrates our proposed map representation can be utilized not only with spherical cameras but also with perspective cameras without requiring camera calibration.

4.1 Mapping

The overall pipeline for the mapping procedure is shown in Figure 4.1. The implemented framework can operate on sequential images taken from either a live camera stream or a pre-recorded dataset. The framework runs feature-based pose estimation and panoptic segmentation in parallel threads, since the panoptic segmentation runs significantly slower. This design choice ensures that the mapping is not delayed by the slower process of segmentation. The faster pose estimation thread continuously adds incoming nodes and relative constraints, keeping the map updated as the robot moves. Meanwhile, segmentation results can be integrated later for loop closure without compromising the real-time performance of the system.

For each new image, we maintain a temporal window of the five most recent frames to initialize new nodes in the graph. The number of frames is selected by empirical test-

ing as a value that yields stable node initialization without excessive computational time. Extracted features are used to estimate the essential matrix via the 8-point algorithm combined with RANSAC. The relative transformations obtained from the pose estimation are added as constraints in the pose graph optimization process. After optimization, if the system detects nodes that are spatially close, these are fed back into the pose estimation stage as potential neighbors, improving the graph’s consistency.

In parallel, the second thread performs panoptic segmentation using the OneFormer [45] image segmentation model. OneFormer is a recent transformer architecture that unifies semantic, instance, and panoptic segmentation within a single framework. OneFormer was chosen because it provides state-of-the-art performance on segmentation tasks. In practice, we observed that OneFormer performs robustly on spherical images, producing consistent segmentation masks even in highly distorted image regions, without fine-tuning. Although this module is slower, it is not critical for constructing the initial graph. Instead, the output of this thread enables loop closures, which can update the map as they become available. Panoptic masks are combined with the corresponding keypoint features to build object descriptors. These local object descriptors of a single frame are compared to a global database of previously seen objects, where associated masks are updated through descriptor aggregation.

Loop closure candidates are identified by the VPR module using associated objects. Once a candidate is detected, the corresponding image pair is sent to feature matching to confirm the closure and proceed with the pose estimation. The resulting constraints are then integrated into the pose graph to recover from accumulated drift and refine the map.

The panoptic segmentation masks can be further utilized for identifying dynamic objects, which improves the robustness of the system. However, enabling this filtering step requires waiting for the segmentation output before feature matching can proceed. As a result, segmentation becomes the bottleneck, and the two modules no longer operate fully in parallel. In practice, the framework can be configured to either prioritize real-time operation by running the threads asynchronously or prioritize robustness by synchronizing them.

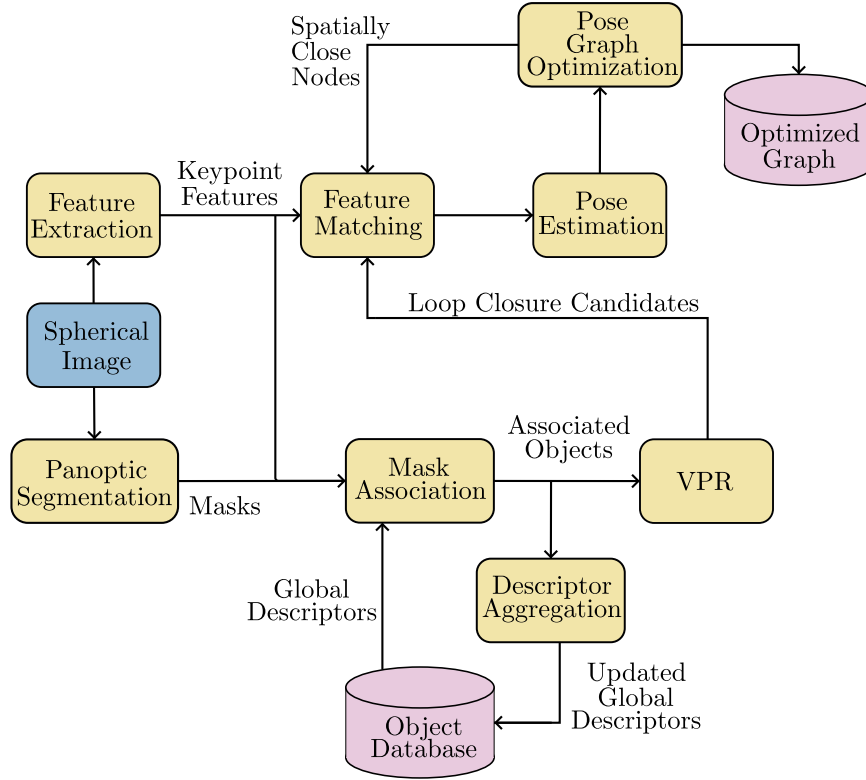


Figure 4.1: Flowchart of the proposed mapping pipeline.

4.1.1 Hierarchical Map Representation

The mapping framework is extended with a hierarchical representation that overlays object-level information onto the underlying pose graph. In this representation, nodes of the pose graph capture camera poses, while objects are anchored to their estimated spatial positions within the map. Each object entry in the global database stores not only its aggregated descriptor but also the list of node IDs and mask IDs corresponding to all of its observations. This explicit linkage between objects and their corresponding nodes provides a connectivity between two layers of the hierarchical map.

We project the centroids of panoptic segmentation masks onto the map using the optimized camera poses obtained from the pose graph to localize objects. The horizontal pixel coordinates of each object centroid can be directly converted into yaw angles in the local reference frame of the corresponding node. For each instance of the object, a ray is cast from the corresponding camera node in the direction of the object, de-

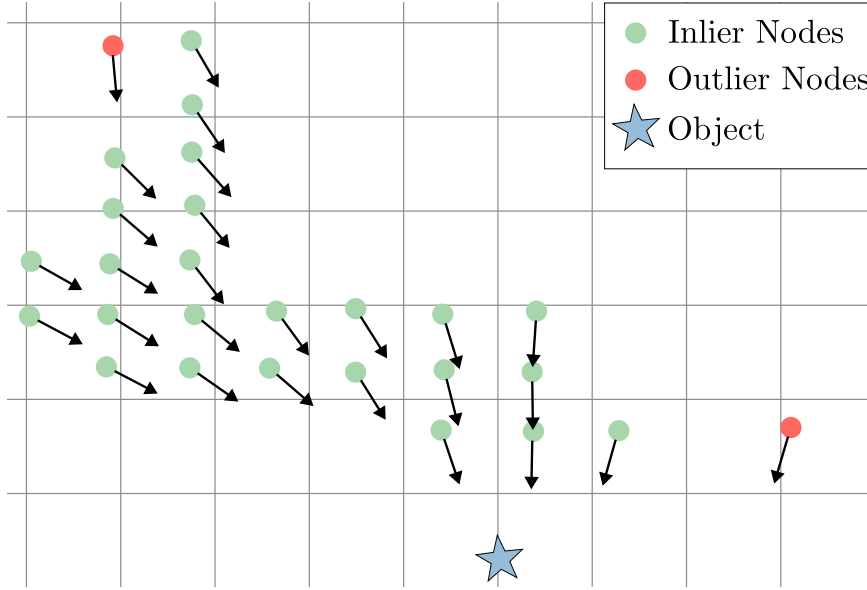


Figure 4.2: Centroid-based object localization using multiple observations.

finer by the yaw angle. Since the same object is observed from multiple nodes, the intersection of these rays provides an estimate of the object's position in the map.

Since noise in feature detection and segmentation can lead to inaccurate or contradictory ray intersections, we adopt a RANSAC-based refinement procedure. In each iteration, a random subset of rays is selected, and their intersection is computed to obtain an object location hypothesis. The number of rays conforming to this hypothesis within a predefined tolerance is counted. The candidate with the largest inlier support is chosen as the final object position. If a significant number of outliers remains after this step, the inliers corresponding to the first detected object are removed, and the RANSAC process is repeated on the remaining rays. This iterative RANSAC implementation allows the system to separate visually similar masks that are incorrectly associated, using the spatial relations of the nodes. An illustrative example of the centroid projection process is shown in Figure 4.2. Each node projects the centroid of its associated object mask into the map coordinate frame, represented by an arrow. The intersection of consistent nodes, shown in green, determines the estimated object position, while inconsistent nodes, shown in red, are treated as outliers and excluded.

4.2 Navigation with 360-Degree View Camera

We implemented a navigation algorithm that operates on the dense pose graph of our hierarchical map. The framework is designed to operate on any differential drive robot platform equipped with a spherical camera, and supports two navigation modalities:

- **Image-goal navigation:** In this modality, the robot is given a target spherical image. The navigator begins by localizing the current observation in the map, identifying the closest node to the current observation using the VPR method described in Section 3.3.4. The target image is then localized in the same manner and added to the graph as a new node. Once both start and goal nodes are localized, the planner computes a route between them using the A* search algorithm.
- **Object-goal navigation:** In this modality, the target is specified as an object within the environment. Here, the object map overlaid on the graph is used to determine the nearest node associated with the target object. Object-goal navigation allows the robot to navigate towards semantically meaningful locations for possible higher-level tasks.

After planning, the robot executes the route incrementally. At each step, the next node along the path is selected as the local target. The robot follows a reactive navigation strategy that continuously re-evaluates the robot's heading and alignment with respect to the target node using spherical image observations. This strategy has an inherent advantage for recovering from drift. The robot computes a heading adjustment by comparing the current observation's SuperPoint features with those of the target node, producing both a similarity score and a heading estimate. If the normalized similarity score exceeds the threshold of 0.9, the local target node is considered reached. The navigation algorithm advances to the next node in the planned path until the goal node is reached. If the heading error exceeds approximately 5 degrees, the robot performs an incremental rotation to align its heading towards the target node. Otherwise, it takes a forward step in the current heading direction. To provide a clearer overview of the proposed navigation algorithm, the overall architecture is depicted in Figure 4.3.

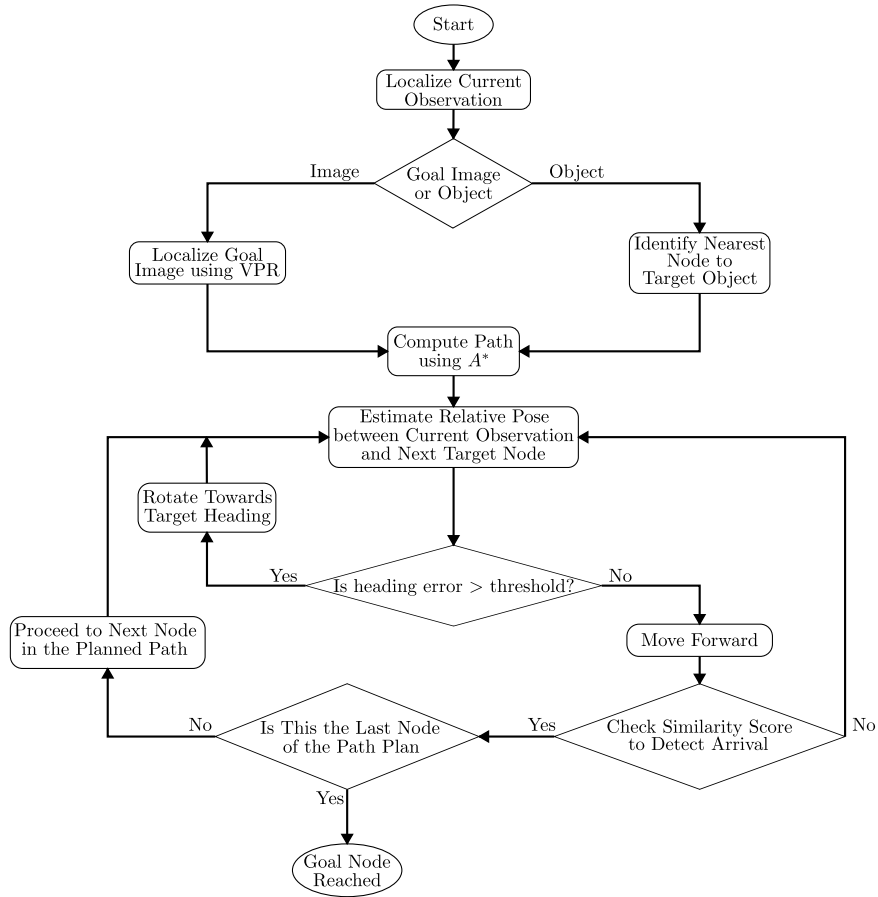


Figure 4.3: Flowchart of the proposed navigation framework, illustrating image-goal and object-goal navigation modalities for spherical images.

4.3 Navigation with Narrow Field of View Camera

The key motivation for this navigation algorithm is that the generated pose graph map is not limited to just spherical cameras. Once a map is constructed using spherical images, it can be used to localize and navigate any robot platform equipped with a camera, regardless of its FOV or camera intrinsics. This makes our framework applicable to a variety of agents equipped with different types of cameras, including uncalibrated perspective cameras.

Implementing standard pose estimation methods, such as 8-PA, is not straightforward when working with perspective images of unknown or ambiguous FOV. These algorithms require projection equations to compute unit direction vectors from pixel coordinates, which rely on the intrinsic camera parameters. Without proper calibration, relative pose estimation between the current and a target frame becomes unreliable. We propose to take advantage of the full 360-degree view of spherical images, which captures all possible viewing directions from a graph node. In our framework, relative transformations between nodes are already computed and are not affected by the unknown camera intrinsics. Given a graph whose nodes are spherical images, localizing a perspective image is a matter of finding the node with the highest visual similarity. Since each spherical image encompasses a complete 360-degree view, we can identify the viewing direction of the perspective camera within that spherical image by aligning the local view of the observation to the global map. The VPR method described in this thesis work is generalizable to perspective images. Therefore, object representations can be utilized to establish candidate nodes in the pose graph. Subsequently, we can determine both the correct spherical node and the viewing direction by performing feature matching between the current observation and candidate graph nodes.

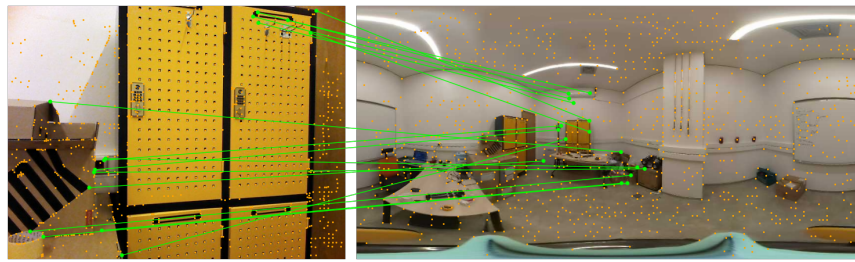
Once we localize the current observation in the map, navigation can be treated as a graph traversal problem. A path leading towards the target can be computed using the A^* search algorithm. The agent can navigate using the known spherical image transformations without needing to estimate any transformations for perspective images. This makes our approach applicable to a range of cameras, eliminating the need for camera calibration.

To correct the robot’s heading in real time, the algorithm continuously performs feature matching between incoming observations and the next spherical node in the pose graph. This provides a reactive navigation strategy that can recover from errors in heading estimation or actuation.

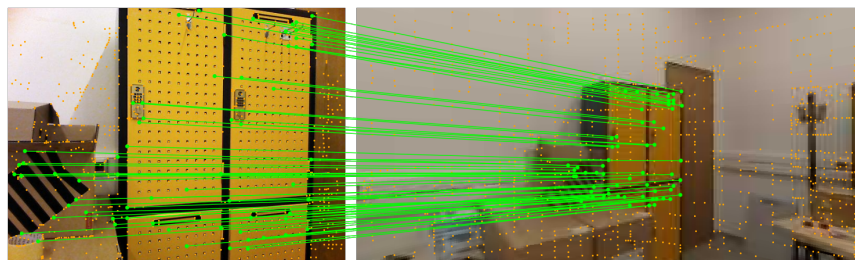
Matching keypoint features between a perspective image and an equirectangular image is challenging due to the distortions introduced by equirectangular projections. These distortions, which are more pronounced near the poles, alter the spatial relations between local features. Therefore, directly comparing an equirectangular image with a perspective image often results in poor matching performance. We propose to use tangent images derived from the spherical images for the feature matching task. Tangent images are perspective projections of a portion of the spherical image in a specific viewing direction. Conceptually, this can be imagined as placing a virtual camera at the center of the sphere, pointing in that direction. By using tangent images, it is possible to create local perspective images with the intended FOV and orientation. The resulting images minimize distortions and improve the robustness of feature matching by approximating perspective views.

Figure 4.4 compares keypoint matching results for a perspective image against two different projections of the same scene. In Figure 4.4a, the perspective image is matched directly to an equirectangular image. Although the visual similarity is evident, the number of detected matches is low, and almost all correspond to incorrect feature pairings. Figure 4.4b shows the matching between the same perspective image and a tangent image extracted from the equirectangular image in a direction aligned with the perspective image’s camera orientation. In this case, the number of matches is notably higher, and the correspondences are highly accurate. This demonstrates how tangent images offer a significant advantage when finding correspondences by reducing distortions. For this reason, we adopt tangent images in our implementation during downstream localization and navigation tasks.

The navigation routine begins by localizing both the start and goal frames in the pose graph. Localization utilizes our novel VPR module to return potential nodes that share common object or objects with the query image. Then, the most visually similar node among the potential nodes is selected based on the number of matching features. If



(a)



(b)

Figure 4.4: Keypoint matching between a perspective image and two different projections of the same scene: (a) equirectangular projection and (b) tangent projection.

either the start or target frame cannot be localized, navigation is aborted. Once start and goal nodes are identified, the algorithm computes an optimal path between them using the A* search algorithm.

Since the robot's initial position is unlikely to coincide exactly with the start node in the pose graph, the system must first navigate to that node before path execution. The ambiguity in the FOV of the perspective camera complicates the heading estimation from the current observation to the start node. The current observation could have been captured either from far away with a narrow FOV or from closer with a wide FOV. To resolve this ambiguity, the algorithm performs an initial heading and FOV calibration phase.

In this phase, the algorithm estimates the heading from the current observation toward the start node by using the spherical image associated with that node. Tangent image projection is generated in the direction aligned with the current observation's estimated viewing direction. Keypoint features are extracted from both the perspective image and the tangent image, and feature matching is used to estimate the proportion of the spherical image that the perspective view covers and to correct the estimated viewing direction. The robot rotates a full 360° with small increments, repeating this process at each step. At each orientation, the algorithm calculates the angular width of the window on the spherical image that corresponds to the current perspective observation.

To visualize, imagine a unit sphere with the spherical camera located at its origin, while the perspective camera is positioned slightly offset from this center. When the perspective camera's viewing direction points away from the sphere's origin, it remains between the viewed scene and the origin, resulting in a narrow angular width on the spherical image. Conversely, when oriented toward the origin, the perspective camera is positioned behind the origin relative to the scene, resulting in coverage of a much larger portion of the spherical image. Figure 4.5 illustrates this concept in a 2D projection, where the sphere is represented as a circle, the camera location is fixed, and the change in covered arc length s is shown for two different viewing directions while the field of view θ the camera remains constant.

After the calibration, the algorithm eliminates the ambiguity in the FOV of the cam-

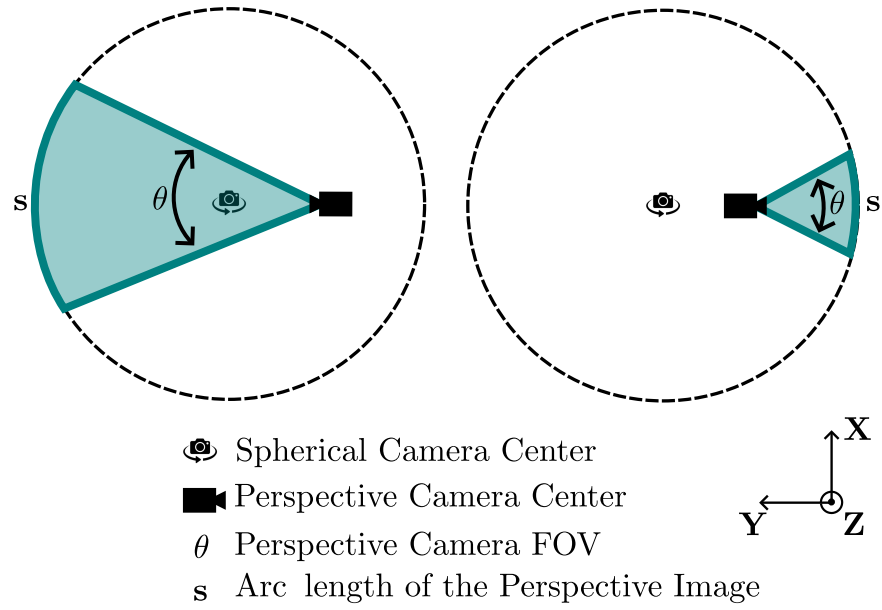


Figure 4.5: 2D projection illustrating the relationship between a perspective camera and the spherical image. The spherical camera is at the circle's center, while the perspective camera is positioned slightly offset from it. The camera's field of view θ is constant, but the corresponding arc length s on the spherical image changes with viewing direction.

era, and the robot knows which way to face to reach the start node in the graph. From there on, the robot follows a loop where it aligns its heading towards the next node and moves forward until it reaches the target. At each step, a tangent image from the target node is generated, which corresponds to the expected perspective image if the camera arrives at the node. The tangent image is used for calculating a heading error and a similarity score. If the heading error exceeds a certain threshold, the robot performs a corrective rotation before attempting forward motion. If the heading alignment is acceptable, the robot moves towards the target node and checks the visual similarity between the current observation and the target tangent image. If the similarity score indicates arrival at the node, the current node is removed from the path plan, and the robot repeats the same process for the next target node. This loop continues until the goal node is reached.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Simulation Environment

The proposed visual SLAM pipeline comprises several components that require extensive testing to inform design choices and evaluate the performance and robustness of the algorithms. The tests during the initial development phase are carried out in a simulated environment using Habitat-Sim [46]. Habitat-Sim supports photorealistic 3D simulation, large-scale scene traversal, configurable sensors, and generic handling of 3D datasets.

There were three main motivations for preferring a simulation environment for evaluation before real-world tests. First of all, conducting extensive real-world experiments would require significant manual labor to set up the environment, collect data, and annotate object masks. Performing all these tasks on a large scale for statistical analysis would be both time-consuming and open to human error. Secondly, the lack of annotated ground truth data, such as object positions, segmentation masks, and camera trajectories, requires additional equipment and effort for precise quantitative assessment of the developed algorithms. In contrast, simulation environments provide highly accurate ground-truth data with minimal effort. Lastly, the real-world environments are dynamic, and their conditions are difficult to control. Especially during the development phase, testing algorithms in a static environment with ideal lighting conditions is essential to obtain precise and reproducible experimental results.

The main problem with robotic simulation is obtaining photorealistic observations. To address this, we used the HM3D (Habitat-Matterport 3D) dataset, which is currently one of the most comprehensive sources of 3D indoor environments. HM3D is

a large-scale dataset of 1,000 3D reconstructions from real-world building interiors with diverse room layouts and accurate semantic labeling. An example scene from the HM3D dataset is shown in Figure 5.1. It outperforms existing photorealistic datasets in terms of scale, completeness of reconstruction, and visual fidelity [47]. This makes it particularly suitable for testing vision algorithms in realistic indoor settings.

In our setup, the agent is equipped with a 360-degree panoramic RGB camera that captures an omnidirectional view of the environment. The panoramic image is generated through equirectangular projection, natively supported by the Habitat-Sim framework with a resolution of 1024x512 pixels. The simulation framework also enables the batch extraction of images from arbitrary locations in the scene. This allows us to collect a large amount of data for extensive evaluation of our methods across a variety of scenes.



Figure 5.1: Dollhouse view of an example HM3D dataset environment

5.2 Feature Extraction and Matching Algorithm Comparison

In this section, we compare the performance of five combinations of feature extraction and matching algorithms on the downstream task of pose estimation. We aim to assess how well each algorithm performs under spherical distortions introduced by equirectangular projections, and to find the most suitable for integration into our visual SLAM framework. We extract 288 equirectangular image pairs from the simulation environment to create a comprehensive benchmark. For each image pair, one is fixed as the origin position, and the other image (taken from the target position) is rotated to simulate 36 different camera orientations, resulting in a total of 10,368 image pairs. This setup allows us to systematically evaluate performance across a wide range of relative poses, with variable heading direction and camera orientation.

For each image pair, pose estimation is carried out using the pipeline described in Section 3.1. We evaluate the performance on two metrics: the absolute heading angle error, which measures the angular difference between the estimated and ground-truth vector pointing from the first camera position to the second, and the absolute yaw error, which quantifies the error in the camera orientation on the Z-axis. These metrics are computed using the known ground-truth camera poses available from the simulation.

We compare five commonly used combinations of feature extraction and matching algorithms. The first one is the combination of learning-based models SuperPoint and LightGlue. The second method uses the classical SIFT descriptor matched with the Brute-Force matcher. The third and fourth methods employ ORB features, also used in popular vSLAM algorithms. The latter one replaces the Brute-Force matcher with the FLANN-based matcher. The fifth algorithm is SPHORB, which uses ORB features adapted specifically for spherical distortions.

The comparative performance of these methods for heading estimation is illustrated in Figure 5.2. Each group of bars shows the percentage of image pairs with heading estimation errors below four thresholds. The bar lengths indicate the percentage of test samples below each threshold. Across all thresholds, the SuperPoint+LightGlue combination consistently outperforms the other methods, with a significant margin.

In contrast, methods based on classical ORB features show the worst performance despite their computational efficiency. SIFT features perform slightly better, but still fall short, around one fourth of samples achieving heading errors under the $<5^\circ$ threshold. SPHORB narrows the gap between classical and learning-based approaches, showing reduced errors compared to ORB and SIFT, though still falling behind SuperPoint+LightGlue. These results highlight the robustness to spherical distortions and suitability for precise heading estimation of modern learned features and attention-based feature matchers.

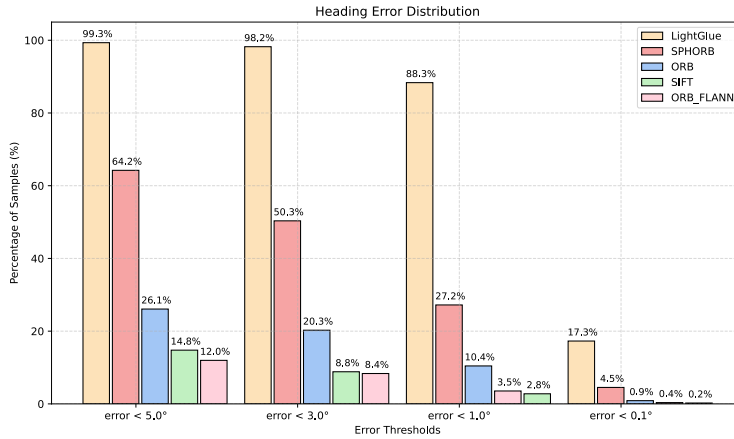


Figure 5.2: Comparison of heading error performance across five feature extraction and matching methods.

Table 5.1 summarizes the quantitative performance of each method in terms of heading angle error. The results once again confirm the clear superiority of the SuperPoint+LightGlue combination, which not only achieves the lowest mean and maximum errors but also exhibits minimal variance, indicating consistent performance. In contrast, traditional methods such as SIFT and ORB yield significantly higher errors. While ORB remains the most computationally efficient, the trade-off in accuracy is substantial. SPHORB provides a compromise by achieving lower heading errors compared to classical descriptors, particularly in terms of mean error, but at the cost of increased computational overhead. While SPHORB improves the mean and standard deviation of the heading error compared to classical ORB/SIFT, it still fails catastrophically on some extreme cases. It is important to note that the reported execution time for SuperPoint+LightGlue corresponds to runs on a GPU-accelerated device and

may vary significantly depending on the available computational resources. Also, reported execution times include the estimation step, which is significantly higher compared to feature matching and extraction for efficient methods. While the method demonstrates high accuracy and consistency, its real-time applicability is dependent on hardware capabilities.

Table 5.1: Heading angle error statistics for each feature matching method.

Algorithm	Mean Error (°)	Max Error (°)	Std. Deviation (σ)	Execution Time (s)
SP+LG	0.53	17.31	0.88	0.23
SIFT	33.65	179.92	36.41	0.36
ORB	46.90	179.98	51.38	0.19
ORB + FLANN	59.06	179.91	50.98	0.19
SPHORB	10.42	179.99	20.54	0.58

5.3 Information Matrix

We conducted an experimental study on the effect of off-plane rotation components on the accuracy of heading estimation to justify the choice of the adaptive information matrix in pose graph optimization. The previous image pairs collected from the simulation are processed using SuperPoint and LightGlue, and the relative poses are estimated by the visual odometry system. The pitch and roll components are extracted from the 3D rotation matrix and compared to the heading angle estimation error.

Figure 5.3 and 5.4 show boxplots that visualize the correlation between pitch, roll angles, and heading error. In both plots, the vertical axis represents the absolute value of heading error in degrees, discretized into bins of 0.1 degrees. The horizontal axis shows the absolute value of the estimated pitch and roll angles in degrees, respectively.

There is a clear trend indicating that larger pitch and roll angles are associated with larger heading errors. For both figures, the median heading error values remain below 1 degree, with relatively low dispersion. As the absolute pitch and roll values increase

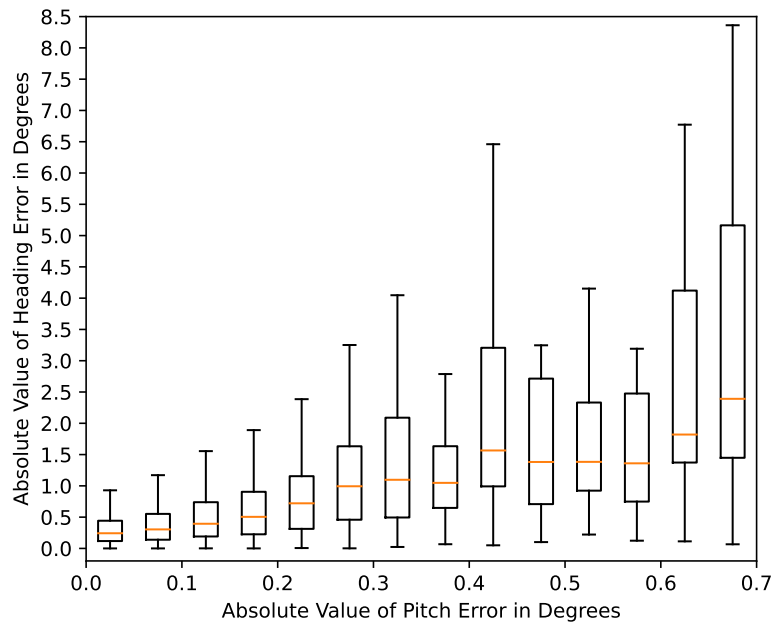


Figure 5.3: Absolute value of pitch angle vs. absolute value of heading error.

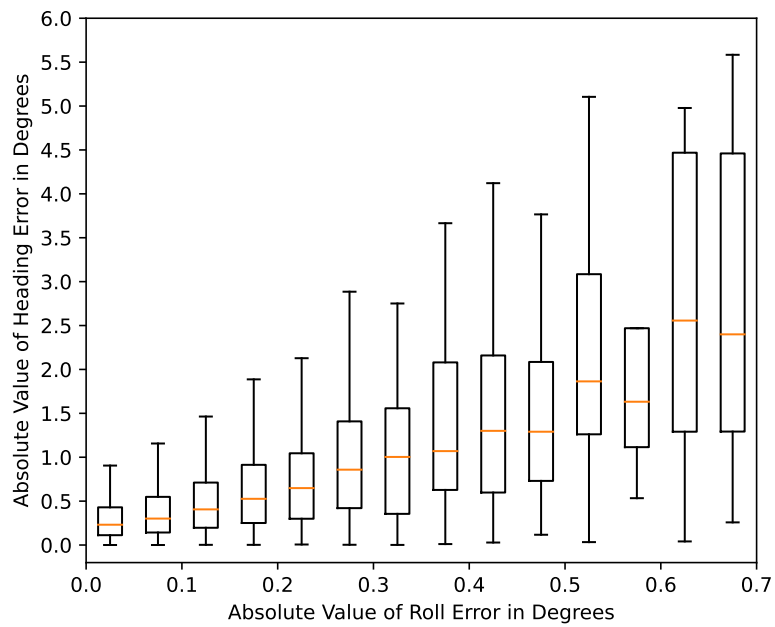


Figure 5.4: Absolute value of roll angle vs. absolute value of heading error.

beyond 0.3 degrees, both the median and variability of the heading error grow with an approximately linear profile. For off-plane rotations above 0.5 degrees, heading errors frequently exceed 2 degrees and display a much wider range, proving that deviations in roll and pitch angles can be linked to heading inaccuracies.

These results confirm that off-plane rotation components provide a reliable indicator for the uncertainty in heading estimation. This empirical correlation justifies the use of an adaptive information matrix in the pose graph optimization, where the confidence assigned to each measurement decreases proportionally to the magnitude of the off-plane rotation components. The optimization becomes more robust to the errors in estimation provided by the visual odometry pipeline.

To evaluate the effect of incorporating the information matrix into pose graph optimization, we conducted another experiment using a simulated indoor environment. Sixteen images were captured from a 4 X 4 grid arrangement with 30 cm spacing between positions. The ground truth poses were defined by simulation coordinates. We compared two configurations: one using a uniform weighting of constraints, another relying on the information matrix derived from estimated rotation consistency.

To assess robustness under degraded visual input, we incrementally reduced the number of keypoints used in the 8-PA for relative pose estimation. Figure 5.5 presents the RMSE of translation error (y-axis in logarithmic scale) as a function of the number of keypoints. With the maximum keypoint count, 500, the RMSE for the information matrix configuration was around 0.15 cm, versus 0.6 cm for uniform weighting. At the lowest tested keypoint count, 25, the RMSE increased substantially for both methods, but remained recognizably lower when the information matrix was applied. These results show that reducing the impact of potentially unreliable measurements based on off-plane rotational deviation helps keep the trajectory accurate, even in difficult visual conditions. In all trials, using the information matrix consistently reduced translational RMSE compared to uniform weighting. As the number of keypoints decreased, the benefit became more pronounced. In this experiment, fewer keypoints cause noisier pose estimates. The ability to assign lower confidence to constraints with larger roll and pitch deviations significantly improved overall map accuracy.

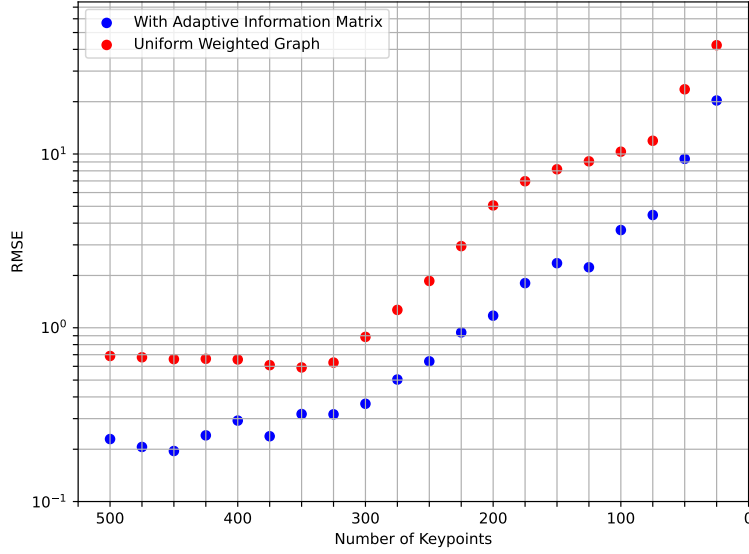


Figure 5.5: RMSE of translation error vs. number of keypoints used in 8-PA for pose estimation.

5.4 Mask Association

For the numerical evaluation of our mask association methods, we formulate the task as a clustering problem and assess performance using three widely used clustering evaluation metrics. The first metric, the Adjusted Rand Index, measures the agreement between two clustering results. In our experiment, these two clustering results correspond to ground-truth object IDs and the clusters produced by our mask association pipeline. Adjusted Rand Index values range from -1 to 1, where 0 corresponds to random chance, and negative values indicate performance worse than chance. The second metric is the Homogeneity score, which measures the purity of the clusters by checking if a given cluster contains only members of a single class. In our setting, it reflects the amount of incorrect object-mask associations. The third metric, Completeness score, measures whether all members of a class are assigned to the same cluster. A low score indicates that masks from the same object are dispersed across multiple clusters, corresponding to missed associations.

To evaluate the two proposed association methods, we collected 144 spherical images

from the HM3D simulation environment. Each image provides ground-truth semantic masks with globally unique object IDs. To ensure reliable evaluation, we restricted the object definition by requiring each segmented mask to contain at least 15 keypoint features and to be observed in at least 3 frames. After filtering, our test set consists of 641 segmentation masks corresponding to 28 distinct objects.

We first examine the effect of the z-score threshold on mask association performance for both the subspace distance and optimal transport methods. Figures 5.6 and 5.7 present the variations of three clustering metrics for a range of threshold values. A higher z-score threshold makes the association more conservative, admitting only highly confident matches. While this increases cluster purity, it also leads to missed associations and over-clustering. This trade-off is visible in both figures. As the threshold increases, homogeneity consistently improves while completeness declines. Although the two methods achieve comparable peak performances, the subspace distance method is more sensitive to changes in the threshold, with performance dropping sharply outside a narrower optimal range. This suggests that the optimal transport method is more robust to parameter tuning. Overall, the best-performing thresholds are determined as 4.4 for subspace distance and 4.8 for optimal transport.

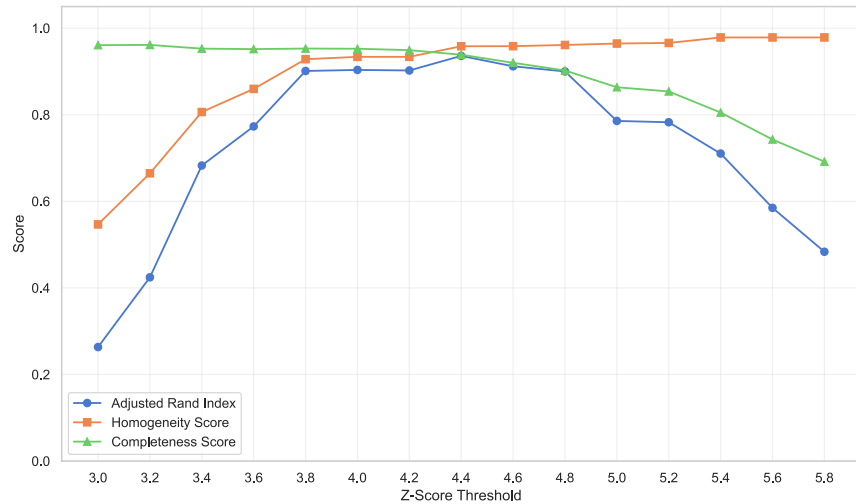


Figure 5.6: Effect of Z-score threshold on mask association for subspace distance-based similarity method.

The clustering accuracy can be further improved by using spatial information about the masks. More specifically, using the constructed pose graph and projecting the

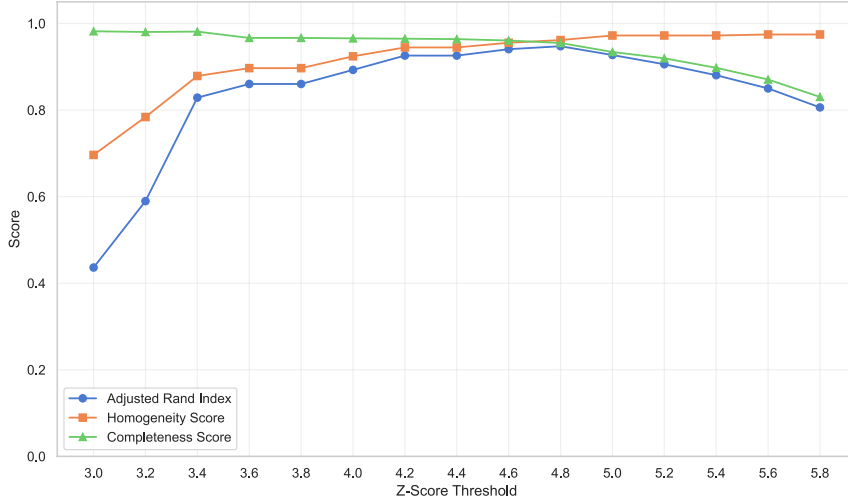


Figure 5.7: Effect of Z-score threshold on mask association for Optimal transport-based similarity method.

mask centroids onto it, we can localize the objects. The iterative RANSAC implementation mentioned earlier enables the detection of mismatched objects with identical visual appearance but different spatial locations, effectively disambiguating them. Figure 5.8 shows an example scenario where the segmentation masks corresponding to two visually identical doors are initially clustered together by the mask association algorithm. In the first iteration of RANSAC, the majority of the masks settle on an object location. However, a large number of masks remain as outliers. In this case, the algorithm removes the inliers of the first iteration from the data and re-runs the RANSAC, resulting in a second object location. Without spatial reasoning, such visually indistinguishable objects would remain grouped together; here, they are correctly separated during object localization.

This object localization step increases the purity of the clusters by eliminating erroneous assignments in the clusters. The peak performances of mask association algorithms are presented in Table 5.2. Optimal transport-based mask association followed by RANSAC refinement almost yields correct clustering. The main reason for the remaining faulty matches is inconsistencies in the predicted segmentation masks.

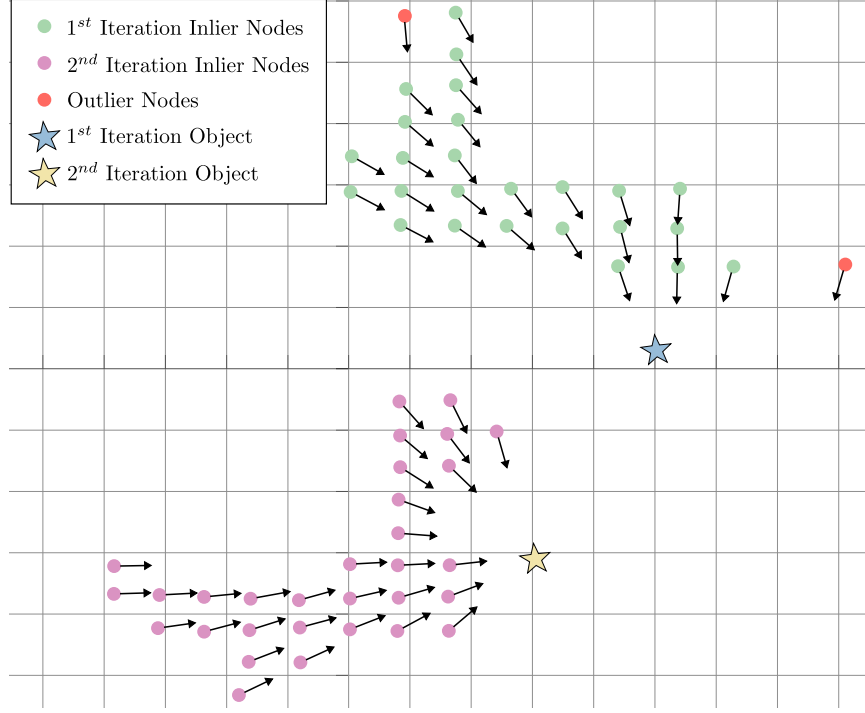


Figure 5.8: Example of the iterative RANSAC separating visually identical objects after mask association.

Table 5.2: The peak performances of mask association algorithms

Algorithm	ARI Score	Homogeneity Score	Completeness Score
Subspace Distance	0.93	0.95	0.92
OT	0.94	0.96	0.95
OT + RANSAC	0.97	0.98	0.97

5.5 Mapping

The proposed mapping procedure is evaluated in three simulated indoor environments of varying size and structural complexity. This setup allows us to assess how well the method generalizes across environments with different spatial layouts and object distributions. In each environment, spherical images were captured on a regular grid to ensure dense coverage of the accessible areas. Topological maps were generated from collected spherical images, in which nodes represent sampled robot poses and edges encode navigational connectivity. Furthermore, the detected object instances were as-

sociated with the corresponding nodes, enabling an object-level map to be projected on the graph. The resulting topological graphs, together with object locations, are presented in Figures 5.9.

The pose graph constructed in the first simulation environment, with the corresponding ground-truth node positions, is shown in Figure 5.9. This environment consists of a single room measuring approximately $3\text{ m} \times 7\text{ m}$, representing a relatively simple indoor setting with minimal structural complexity. Due to strong visibility between sampled poses and a simple geometric layout, the estimated pose graph closely aligns with the ground-truth node distribution. In particular, the central region of the room exhibits a high edge density, as many nodes are mutually visible. This redundancy in connectivity provides stronger geometric constraints, resulting in more accurate optimization outcomes for the overall graph structure.

The second simulation environment, shown in Figure 5.10, is considerably larger and structurally more complex. It consists of a bedroom, an open kitchen, and a living room connected by a long and narrow corridor, covering an area of approximately $10\text{ m} \times 22\text{ m}$. A total of 320 images were collected from this space. In contrast to the first environment, this map introduces additional challenges such as repetitive patterns, low-visibility regions, and extended corridors with reduced geometric constraints.

During early experiments, a failure case was observed in the graph construction process. Specifically, our system relies on a keypoint threshold to decide whether two nodes should be considered neighbors. With the threshold set to 200, a spurious edge was introduced due to visual similarity between two distinct but visually alike bathroom doorways, as illustrated in Figure 5.11. Although the images were taken at different locations, the high visual similarity in a small portion of the images resulted in an incorrect neighbor match.

When incorporated into the pose graph optimization, this faulty edge distorted the overall map structure. As shown in Figure 5.12, the spurious edge marked with a circle causes the pose graph optimization to connect two spatially distant places, resulting in a topologically inaccurate map. Rebuilding the graph with a higher threshold eliminated the erroneous connection and restored the topological accuracy of the map given in Figure 5.10. However, raising the threshold also carries the risk of reject-

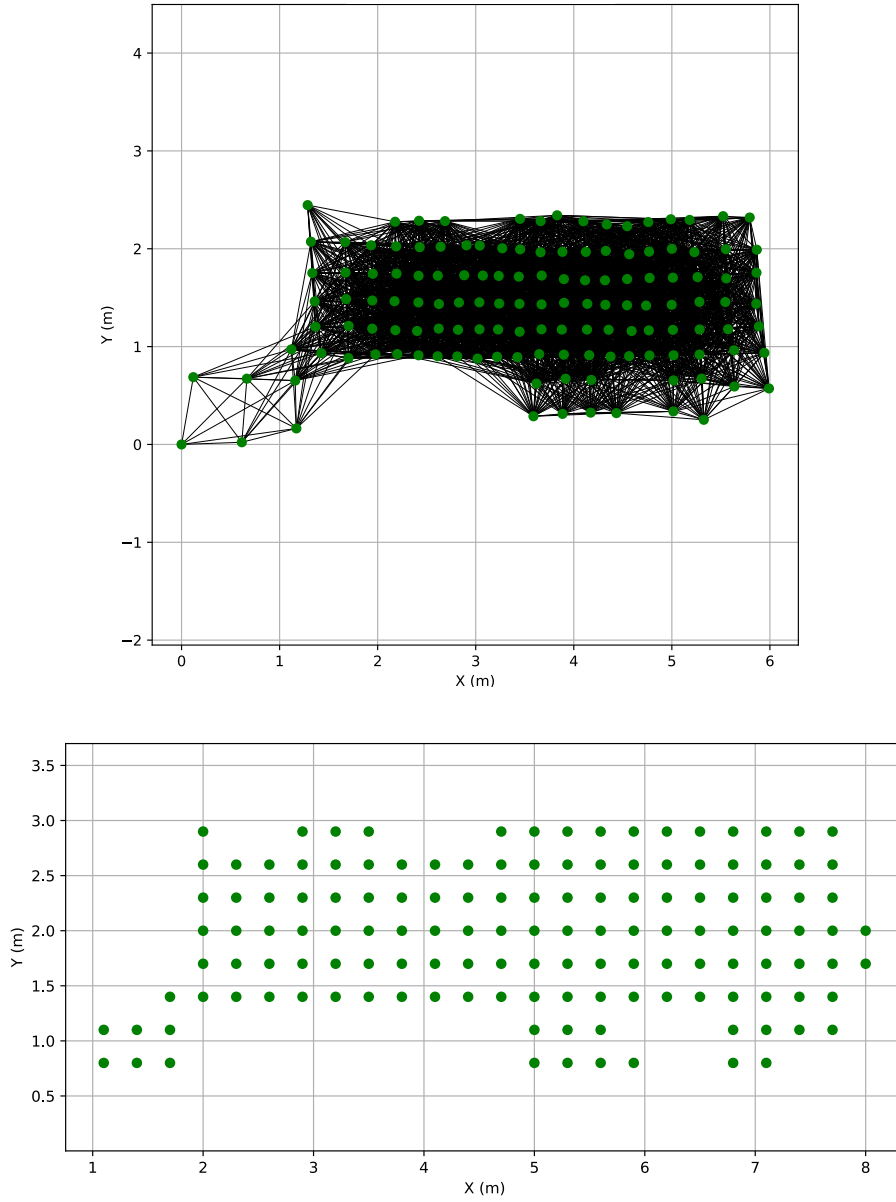


Figure 5.9: Pose graph (top) and corresponding ground-truth node positions (bottom) for the first simulation environment.

ing valid neighboring nodes, potentially reducing graph connectivity. The problem of incorrect matches can lead to a critical failure mode, as they alter the connectivity and perceived traversability of the environment. A more robust approach would be to design the optimization to explicitly handle such spurious edges, for example, by integrating robust cost functions, switchable constraints, or outlier rejection mecha-

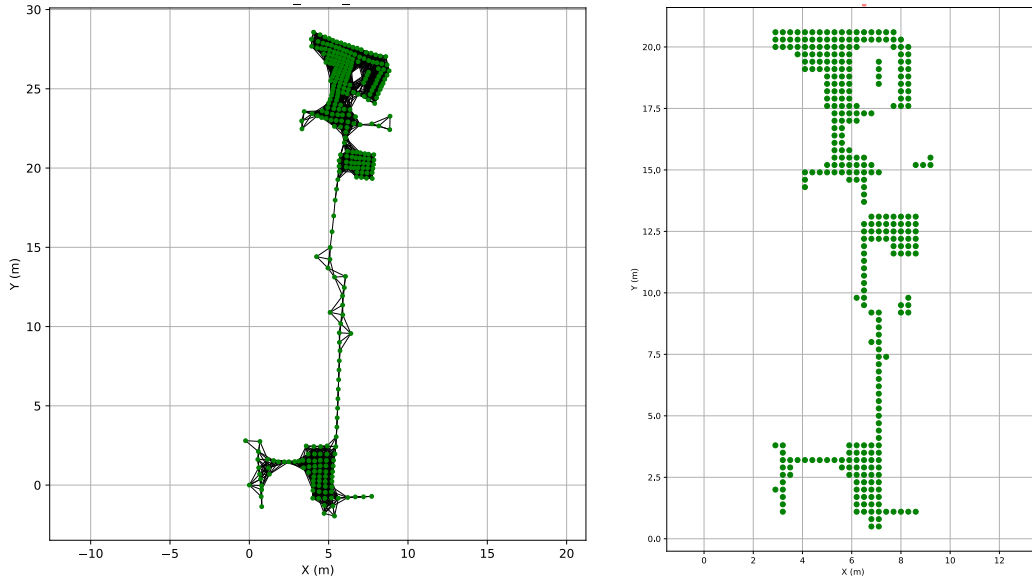


Figure 5.10: Pose graph (left) and corresponding ground-truth node positions (right) for the second simulation environment.

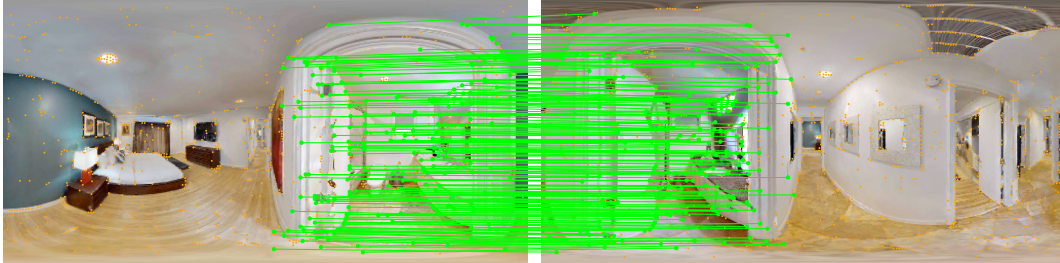


Figure 5.11: Two frames with visually similar bathroom doorways, that were incorrectly matched as neighbors, leading to a spurious edge in the pose graph.

nisms that eliminate inconsistent measurements. For this scenario, even with using a higher threshold, the large scale and structural complexity of the environment results in minor residual distortions. Nevertheless, the pose graph maintains a topologically accurate representation suitable for navigation purposes.

The third simulation environment consists of a bedroom and a living room on an approximately 10 x 12 floor space. This environment is rich with both distinct and

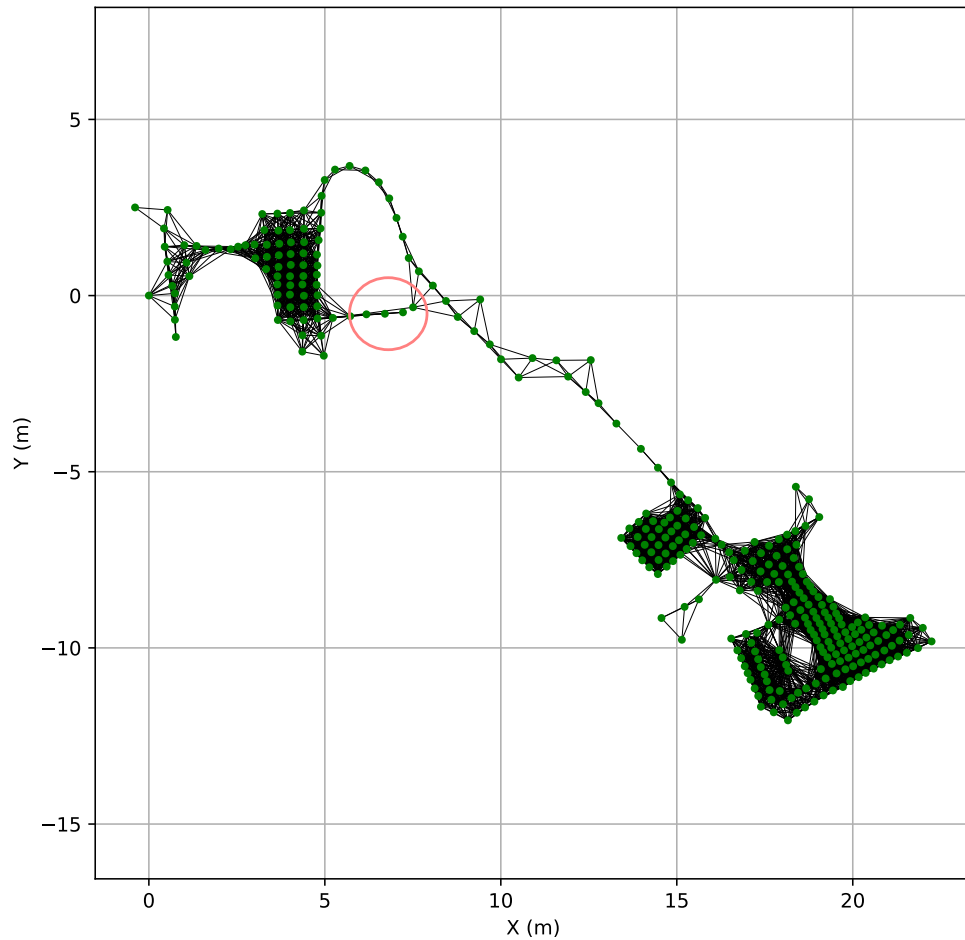


Figure 5.12: A spurious edge, marked on the graph, introduces a faulty connection that distorts the optimized pose graph.

repetitive objects. The pose graph consists of 133 spherical images. Figure 5.13 shows the constructed pose graph at the top and the corresponding ground truth at the bottom. The only remarkable mistake is the edge between nodes 33 and 34 marked in a circle. These two portions of the map are connected only with a single edge, leading to a scale ambiguity. The doorway blocks the visibility among other nearby nodes. Apart from the scale distortion, the map is quite accurate.

Additionally, Figure 5.14 illustrates the constructed pose graph merged with the top-down map of the simulation environment. The graph has been geometrically scaled to align with the real-world dimensions of the map, enabling a direct visual comparison between the robot trajectory and the spatial layout of the environment. The poses where images were captured are marked with green dots, showing the discrete

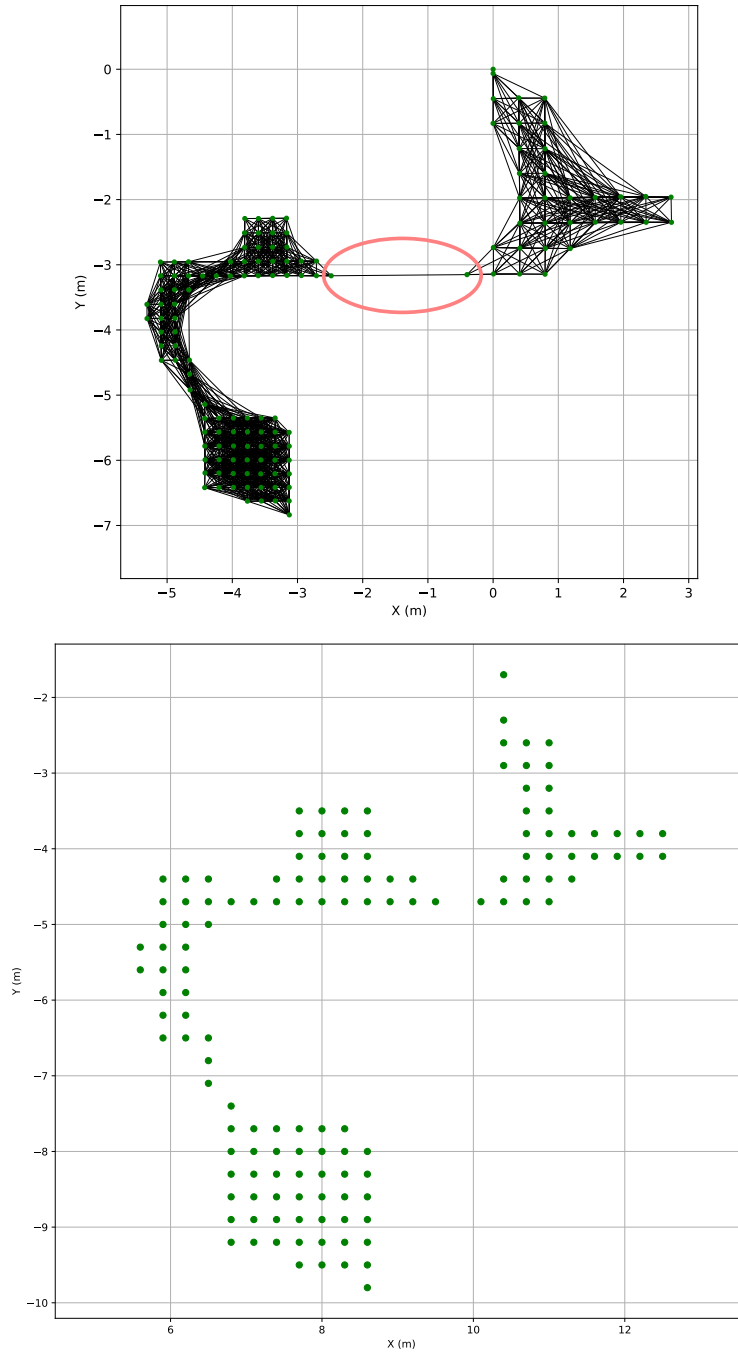


Figure 5.13: Pose graph (top) and corresponding ground-truth node positions (bottom) for the third simulation environment.

sampling of the environment during traversal. Red dots correspond to the estimated object locations, obtained through the proposed mask association and centroid projection pipeline. As observed, the object projections are spatially consistent with the underlying environment map, demonstrating that the mapping process preserves the

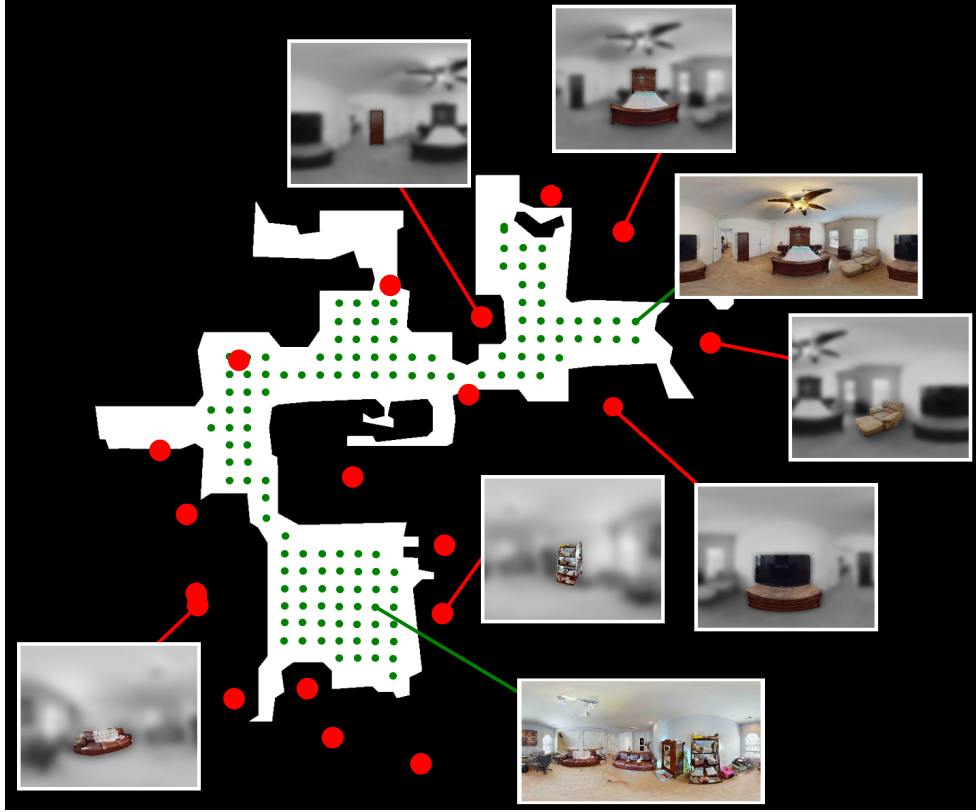


Figure 5.14: Merged representation of the pose graph for the simulation environment and object locations overlaid on top.

relative topology between robot poses and surrounding objects. Overall, this figure provides a clear visualization of how the proposed framework integrates geometric robot trajectories with object-level semantic information.

5.6 Navigation

After constructing the topological maps, we evaluate their performance by conducting navigation experiments. Specifically, we consider a set of navigation scenarios where the robot is tasked to traverse between different start and goal positions in the simulated environment. The scenarios were chosen to cover both medium and long-range navigation goals, including room transitions and traversals through narrow passages. The robot’s motion was simulated with discrete steps of 5° for rotations and 10 cm translations. A total of 15 navigation scenarios were performed, each executed on

the constructed topological maps. These experiments test not only the connectivity of the maps but also their effectiveness in supporting localization with Visual Place Recognition and high-level path planning.

The navigation procedure was evaluated in terms of success rate, which is the proportion of scenarios where the target is reached, and the distance error at waypoints, which quantitatively measures the deviations from intermediate nodes along the path plan. These metrics together capture both the reliability of navigation and the accuracy of path following using reactive heading estimation.

Of the 15 scenarios, 14 were completed successfully. A successful navigation execution is illustrated in Figure 5.15. The pose graph is overlaid on the top-down map of the environment. The scale of the pose graph is manually adjusted for visualization. Navigable nodes of the graph are marked by green dots. On the left, the planned path from the start to the goal is shown with a blue line. On the right, the executed trajectory by the robot is drawn on top of it. Notably, the executed trajectory closely follows the planned path, demonstrating the reliability of the navigation algorithm. Only failure occurred during a room transition between the nodes on either side of the door. Although the robot successfully passed this same door in other scenarios, such as in Figure 5.15, in this case, when the robot approached the door frame from a certain angle, the narrow geometry of the passage made it difficult to estimate the correct heading. As a result, the robot entered an oscillatory behavior, continuously rotating one step clockwise and one step counter-clockwise without making progress, ultimately becoming stuck in place. This failure mode highlights a limitation in handling precise heading estimation in constrained passages. This observation suggests that while the proposed topological maps enable reliable navigation across diverse environments, additional mechanisms are required to address challenging transitions at narrow doorways. Possible solutions include incorporating local geometric cues or introducing a recovery behavior to escape oscillatory loops. Addressing these issues would further improve the robustness of the system in real-world scenarios.

The distance error at the waypoints across 14 successful scenarios is shown in Figure 5.16. This data is collected during navigation as the robot signals that it has reached a target node in the path plan. The simulation calculates the difference between the

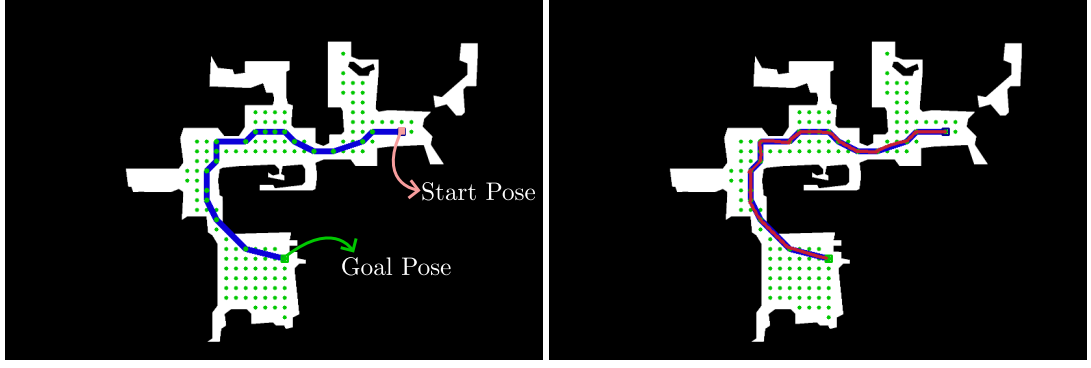


Figure 5.15: Example navigation scenario.

robot’s position and the ground truth position of the target node. Since each trajectory has a different number of waypoints for each scenario, 10 waypoints are evaluated. As shown, the mean position error oscillates around 3 cm, with a maximum deviation from the mean of approximately 4 cm. Importantly, due to the reactive nature of the algorithm, errors do not accumulate along successive waypoints. The error characteristics remain consistent across all waypoint indices. This is particularly important for demonstrating how the proposed reactive navigation algorithm handles cumulative errors over a trajectory, which is critical for long-term autonomy in real-world environments. Although the method does not explicitly target metric accuracy, the results demonstrate that the arrival at the waypoints is detected with high precision, indicating robust performance of the navigation system.

5.7 Real-World Mapping and Visual Place Recognition

For testing the robustness and accuracy of the proposed mapping framework in this thesis, we conducted a series of real-world tests. The first phase of the real-world test includes mapping of METU ROMER laboratories, using a handheld Ricoh Theta X camera. In the first scenario, the trajectory started inside a laboratory room, continued through the corridor following a rectangular loop, and terminated back in the same laboratory room. Figure 5.17 presents the mapping results without the VPR module enabled. The nodes in the resulting pose graph are color-coded to visualize traversal order, ranging from purple at the start of the sequence to yellow at the end. In this case, due to the limited temporal window and accumulated drift, the system

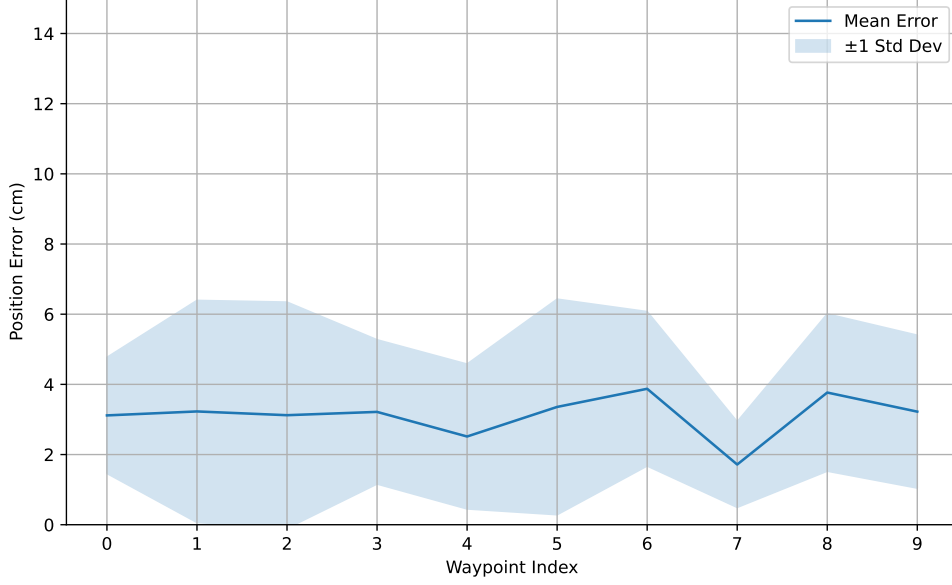


Figure 5.16: Mean distance error for waypoint navigation with shaded variability across 14 scenarios.

fails to associate the revisited laboratory with its initial observation, resulting in the creation of a spurious “second” room. These regions are highlighted with circles in the figure. Since the temporal matching window is short, and the accumulated drift prevents spatial proximity, the algorithm does not attempt to compare the final images with those captured at the beginning of the sequence. On the other hand, Figure 5.18 shows the mapping outcome with the VPR module activated. In this case, loop closures are successfully detected, candidate edges are incorporated into the graph, and the optimization step corrects the drift, effectively merging the duplicated room and restoring the map consistency.

In Figure 5.19, the object locations are overlaid onto the pose graph by projecting the centroids of the corresponding masks. Each green dot corresponds to a pose graph node where a spherical image was captured, while the red dots indicate the projected positions of the detected objects in the map. The masks of some objects are shown as an example. The map contains 32 distinct objects in total, which is considerably fewer than the number of frames in the trajectory. This highlights one of the key advantages of the proposed VPR method. Since our method searches through objects rather than captured frames, for loop closure detection, the search complexity

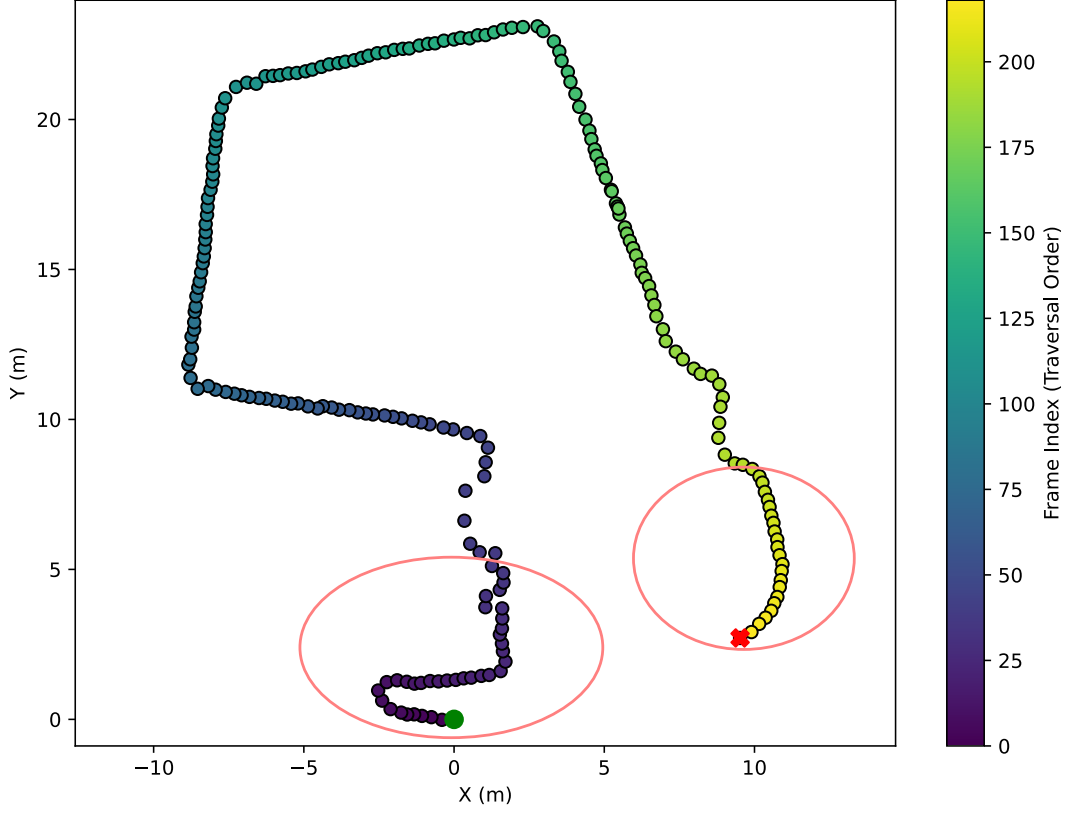


Figure 5.17: Pose graph for real-world scenario 1 without VPR loop closures.

reduces, and node retrieval becomes faster. Overall, the object locations are topologically consistent. Occasionally, visually identical objects such as blackboards in the corridors are incorrectly matched by the mask association pipeline. However, applying iterative RANSAC during centroid projection helps to distinguish these masks by leveraging their spatial relations. A current limitation of the proposed method is that VPR performance relies on the presence of distinctive objects. While object-rich environments such as laboratories provide dense and reliable cues, more uniform areas like corridors contain fewer objects, which reduces the robustness of loop closure detection.

Figure 5.19 also contains some object nodes that are located far from the pose graph nodes, such as one at the bottom of the map. The reason for that faulty localization is the inconsistent mask predictions that our panoptic segmentation model produces. Figure 5.20 portrays an example mask pair taken from two consecutive frames. These masks belong to the desk located at the bottom of the map. In the top frame, the

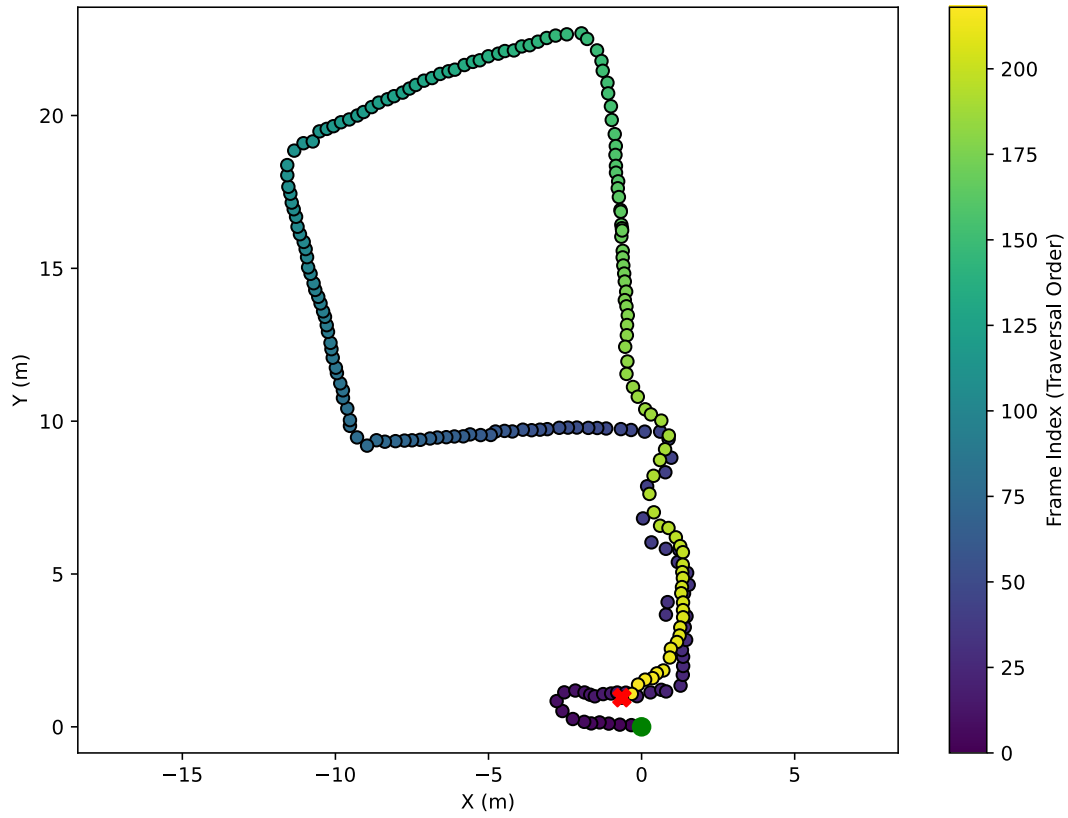


Figure 5.18: Pose graph for real-world scenario 1 with VPR loop closures activated.

segmentation model fails to distinguish the two adjacent desks. As a result, the object centroid shifts, which leads to incorrect triangulation of the object position.

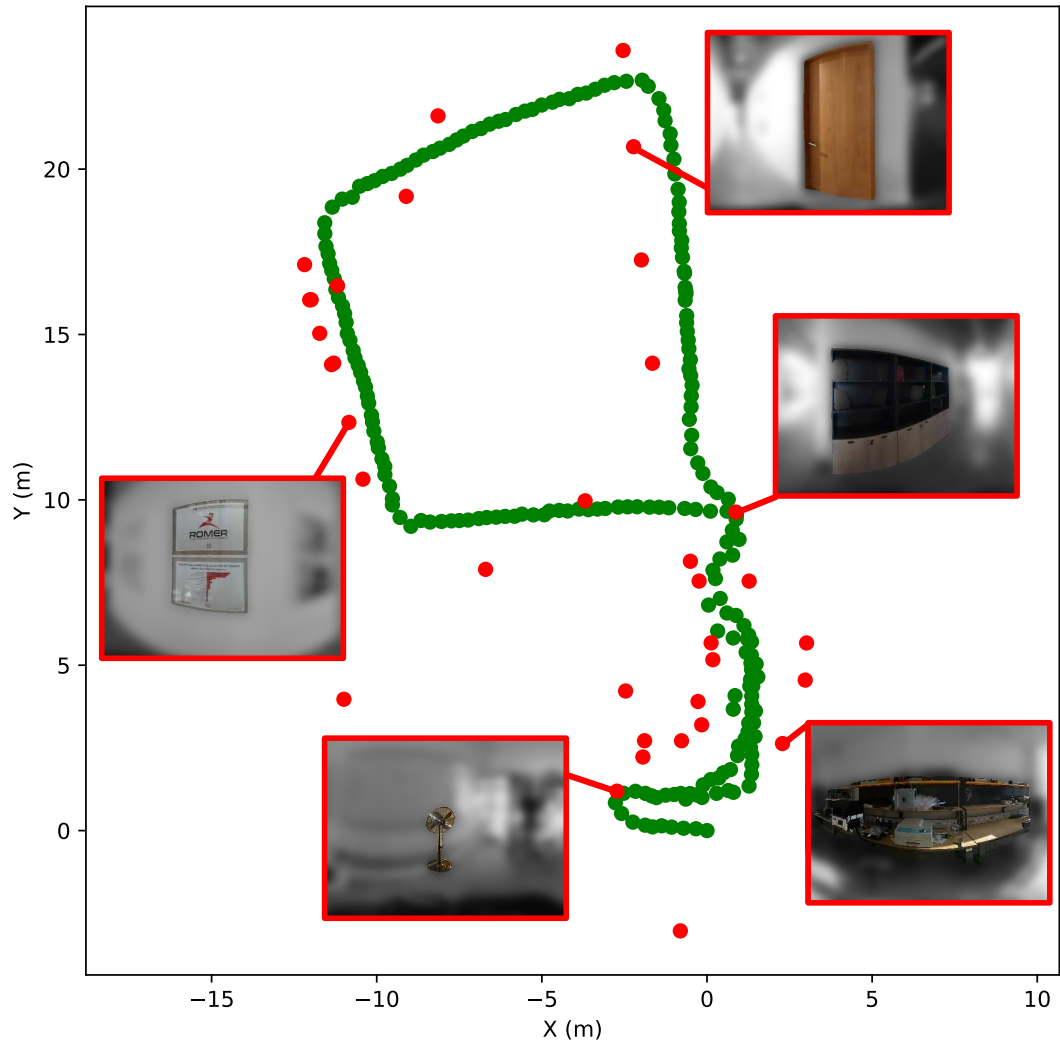


Figure 5.19: Pose graph for real-world scenario 1 with object locations overlaid on top.



Figure 5.20: Inconsistent object masks produced by the panoptic segmentation model.

CHAPTER 6

CONCLUSION AND FEATURE WORK

In this thesis study, an indoor topological navigation framework for spherical images is proposed. The proposed framework leverages the semantic information of the surrounding objects for improving pose estimation accuracy, detecting loop closures, and localizing the robot in the map. For the effective integration of geometric information between robot poses and object-level information, a hierarchical map representation is designed and employed. State-of-the-art feature detection and matching methods are utilized in a relative pose estimation pipeline between two spherical images. A pose graph optimization implementation with an adaptive information matrix is developed from empirical tests. A novel VPR method that relies on co-visible objects in the environment is introduced. Ultimately, the proposed mapping and navigation procedures are rigorously tested in both the simulation environment Habitat-Sim and the real world. The effectiveness of both geometric and semantic layers is experimentally validated. The robots can perform both image-goal and object-goal navigation modalities using the proposed hierarchical map representation. With a loss of small loss of reliability, the constructed topological maps can also be used for navigation by robots equipped with perspective cameras.

There are several promising directions for future work:

- The accuracy and robustness of the proposed framework are strongly dependent on its core perception modules: feature extraction, feature matching, and panoptic segmentation. The models employed for these tasks in the literature are predominantly trained on perspective image datasets. While they demonstrate satisfactory performance when applied to equirectangular images, their lack of spherical awareness introduces distortions and limits generalization.

Future work could explore adapting and training these data-driven models directly on spherical images or designing architectures that explicitly account for panoramic distortions. Such efforts would likely improve both the geometric consistency of feature correspondences and the semantic reliability of scene understanding, thereby strengthening the overall performance of spherical visual SLAM.

- A natural extension of this work is the integration of panoptic segmentation into the navigation process for dynamic scene understanding. Since the framework already identifies and localizes objects, this semantic information can be leveraged to continuously update object positions within the map. Such semantic map evolution would allow the system to remain robust in environments that change over time, reducing reliance on static assumptions. Ultimately, this approach paves the way toward lifelong SLAM, where both geometric and semantic representations are incrementally refined to support long-term autonomy.
- Despite the abundance of perspective camera benchmarks such as TUM RGB-D, only a few small-scale captures with spherical cameras offer true SLAM-style sequences in panoramic format. Their coverage is limited in scale, diversity, and public availability, preventing a comprehensive benchmarking of spherical visual SLAM algorithms. Existing panoramic datasets like HM3D [47] focus on static panoramas for reconstruction and recognition, lacking the temporal continuity required for vSLAM evaluation. This scarcity motivates the generation of new datasets tailored for 360° vSLAM, in which raw equirectangular video is captured alongside accurate ground-truth poses, enabling fair and reproducible evaluation of our proposed method under realistic indoor conditions.
- While our current pipeline for navigation with perspective cameras using SuperPoint + LightGlue enables sparse feature matching between perspective and tangent images derived from equirectangular images, it remains sensitive to mismatches, leading to abrupt failures in estimating the orientation and field of view of the perspective camera. Subsequent studies may focus on several approaches to improve the robustness of the navigation algorithm. Dense match-

ers such as LoFTR[36] or DINOv2[48] could provide higher recall and more uniformly distributed correspondences across the image, reducing reliance on a few keypoints and improving stability. Instead of relying solely on feature matches, future approaches could incorporate robust geometric constraints on the sphere or similarity transforms that do not require knowledge of the camera intrinsics. Outlier rejection schemes, such as RANSAC or angular-error-based filtering, could prevent navigation failures due to a small number of mismatched features.

REFERENCES

- [1] H. F. Durrant-Whyte, *Integration, Coordination and Control of Multi-Sensor Robot Systems*. USA: Kluwer Academic Publishers, 1987.
- [2] H. Durrant-Whyte and T. Bailey, “Simultaneous localization and mapping: part i,” *IEEE Robotics and Automation Magazine*, vol. 13, no. 2, pp. 99–110, 2006.
- [3] S. Thrun, “Probabilistic robotics,” *Commun. ACM*, vol. 45, p. 52–57, Mar. 2002.
- [4] R. Smith, M. Self, and P. Cheeseman, “Estimating uncertain spatial relationships in robotics,” in *Uncertainty in Artificial Intelligence*, vol. 5 of *Machine Intelligence and Pattern Recognition*, pp. 435–461, North-Holland, 1988.
- [5] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse, “Monoslam: Real-time single camera slam,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 6, pp. 1052–1067, 2007.
- [6] M. Montemerlo, S. Thrun, D. Koller, and B. Wegbreit, “Fastslam: a factored solution to the simultaneous localization and mapping problem,” in *Eighteenth National Conference on Artificial Intelligence*, (USA), p. 593–598, American Association for Artificial Intelligence, 2002.
- [7] G. Klein and D. Murray, “Parallel tracking and mapping for small ar workspaces,” in *2007 6th IEEE and ACM International Symposium on Mixed and Augmented Reality*, pp. 225–234, 2007.
- [8] E. Rosten and T. Drummond, “Machine learning for high-speed corner detection,” in *Computer Vision – ECCV 2006*, (Berlin, Heidelberg), pp. 430–443, Springer Berlin Heidelberg, 2006.
- [9] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, “Orb-slam: A versatile and accurate monocular slam system,” *IEEE Transactions on Robotics*, vol. 31, p. 1147–1163, Oct. 2015.

- [10] J. Engel, T. Schöps, and D. Cremers, “Lsd-slam: Large-scale direct monocular slam,” in *Computer Vision – ECCV 2014* (D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, eds.), (Cham), pp. 834–849, Springer International Publishing, 2014.
- [11] J. J. Engel, V. Koltun, and D. Cremers, “Direct sparse odometry,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, pp. 611–625, 2016.
- [12] C. Forster, M. Pizzoli, and D. Scaramuzza, “Svo: Fast semi-direct monocular visual odometry,” in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 15–22, 2014.
- [13] B. Kuipers, “The spatial semantic hierarchy,” *Artificial Intelligence*, vol. 119, no. 1, pp. 191–233, 2000.
- [14] M. Cummins and P. Newman, “Fab-map: Appearance-based place recognition and mapping using a learned visual vocabulary model,” *ICML 2010 - Proceedings, 27th International Conference on Machine Learning*, pp. 3–10, 2010.
- [15] M. J. Milford and G. F. Wyeth, “Seqslam: Visual route-based navigation for sunny summer days and stormy winter nights,” in *2012 IEEE International Conference on Robotics and Automation*, pp. 1643–1649, 2012.
- [16] N. Savinov, A. Dosovitskiy, and V. Koltun, “Semi-parametric topological memory for navigation,” in *International Conference on Learning Representations*, 2018.
- [17] L. Suomela, J. Kalliola, H. Edelman, and J.-K. Kämäräinen, “Placenav: Topological navigation through place recognition,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [18] R. R. Wiyatno, A. Xu, and L. Paull, “Lifelong topological visual navigation,” *IEEE Robotics and Automation Letters*, vol. 7, pp. 9271–9278, 2021.
- [19] R. F. Salas-Moreno, R. A. Newcombe, H. Strasdat, P. H. Kelly, and A. J. Davison, “Slam++: Simultaneous localisation and mapping at the level of objects,” in *2013 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1352–1359, 2013.

- [20] J. McCormac, A. Handa, A. Davison, and S. Leutenegger, “Semanticfusion: Dense 3d semantic mapping with convolutional neural networks,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4628–4635, 2017.
- [21] M. Runz, M. Buffier, and L. Agapito, “Maskfusion: Real-time recognition, tracking and reconstruction of multiple moving objects,” in *2018 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pp. 10–20, 2018.
- [22] G. Narita, T. Seno, T. Ishikawa, and Y. Kaji, “Panopticfusion: Online volumetric semantic mapping at the level of stuff and things,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, p. 4205–4212, IEEE Press, 2019.
- [23] L. Nicholson, M. Milford, and N. Sünderhauf, “Quadricslam: Dual quadrics from object detections as landmarks in object-oriented SLAM,” *IEEE Robotics Autom. Lett.*, vol. 4, no. 1, pp. 1–8, 2019.
- [24] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, “Kimera: an open-source library for real-time metric-semantic localization and mapping,” *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1689–1696, 2019.
- [25] D. Chaplot, R. Salakhutdinov, A. Gupta, and S. Gupta, “Neural topological slam for visual navigation,” *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 12872–12881, 2020.
- [26] S. Garg, K. Rana, M. Hosseinzadeh, L. Mares, N. Sünderhauf, F. Dayoub, and I. Reid, “Robohop: Segment-based topological map representation for open-world visual navigation,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4090–4097, 2024.
- [27] C. Estrada, J. Neira, and J. Tardos, “Hierarchical slam: real-time accurate mapping of large environments,” *IEEE Transactions on Robotics*, vol. 21, no. 4, pp. 588–596, 2005.
- [28] M. Bosse, P. Newman, J. Leonard, M. Soika, W. Feiten, and S. Teller, “An atlas framework for scalable mapping,” in *2003 IEEE International Conference on*

Robotics and Automation (Cat. No.03CH37422), vol. 2, pp. 1899–1906 vol.2, 2003.

- [29] N. Tomatis, I. Nourbakhsh, and R. Siegwart, “Hybrid simultaneous localization and map building: a natural integration of topological and metric,” *Robotics and Autonomous Systems*, vol. 44, no. 1, pp. 3–14, 2003.
- [30] J.-L. Blanco, J.-A. Fernández-Madrigal, and J. González, “Toward a unified bayesian approach to hybrid metric–topological slam,” *IEEE Transactions on Robotics*, vol. 24, no. 2, pp. 259–270, 2008.
- [31] P.-E. Sarlin, C. Cadena, R. Siegwart, and M. Dymczyk, “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12708–12717, IEEE Computer Society, 2019.
- [32] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese, “3d scene graph: A structure for unified semantics, 3d space, and camera,” in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 5664–5673, 2019.
- [33] Q. Zhao, W. Feng, L. Wan, and J. Zhang, “Sphorb: A fast and robust binary feature on the sphere,” *International Journal of Computer Vision*, vol. 113, no. 2, pp. 143–159, 2015.
- [34] J. Cruz-Mota, I. Bogdanova, B. Paquier, M. Bierlaire, and J. Thiran, “Scale-invariant feature transform on the sphere: Theory and applications,” *International Journal of Computer Vision*, vol. 98, no. 2, pp. 217–241, 2012.
- [35] D. DeTone, T. Malisiewicz, and A. Rabinovich, “Superpoint: Self-supervised interest point detection and description,” *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 337–33712, 2017.
- [36] J. Sun, Z. Shen, Y. Wang, H. Bao, and X. Zhou, “Loftr: Detector-free local feature matching with transformers,” in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8918–8927, 2021.

- [37] G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, “A tutorial on graph-based slam,” *IEEE Intelligent Transportation Systems Magazine*, vol. 2, no. 4, pp. 31–43, 2010.
- [38] D. Gálvez-López and J. D. Tardós, “Bags of binary words for fast place recognition in image sequences,” *IEEE Transactions on Robotics*, vol. 28, pp. 1188–1197, October 2012.
- [39] R. Arandjelović, P. Gronat, A. Torii, T. Pajdla, and J. Sivic, “Netvlad: Cnn architecture for weakly supervised place recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1437–1451, 2018.
- [40] A. Ali-Bey, B. Chaib-Draa, and P. Giguère, “Mixvpr: Feature mixing for visual place recognition,” in *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 2997–3006, 2023.
- [41] G. Berton, G. Trivigno, B. Caputo, and C. Masone, “Eigenplaces: Training viewpoint robust models for visual place recognition,” in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 11046–11056, 2023.
- [42] S. Izquierdo and J. Civera, “Optimal transport aggregation for visual place recognition,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 17658–17668, 2024.
- [43] A. Wang, H. Chen, L. Liu, K. CHEN, Z. Lin, J. Han, and G. Ding, “YOLOv10: Real-time end-to-end object detection,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [44] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” in *Advances in Neural Information Processing Systems*, vol. 28, Curran Associates, Inc., 2015.
- [45] J. Jain, J. Li, M. Chiu, A. Hassani, N. Orlov, and H. Shi, “OneFormer: One Transformer to Rule Universal Image Segmentation,” in *CVPR*, 2023.
- [46] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra, “Habitat: A platform for embodied ai research,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9338–9346, 2019.

- [47] K. Yadav, R. Ramrakhya, S. K. Ramakrishnan, T. Gervet, J. A. Turner, A. Gokaslan, N. Maestre, A. X. Chang, D. Batra, M. Savva, A. W. Clegg, and D. S. Chaplot, “Habitat-matterport 3d semantics dataset,” *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4927–4936, 2022.
- [48] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. HAZIZA, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, “DINOv2: Learning robust visual features without supervision,” *Transactions on Machine Learning Research*, 2024.